



Streaming, Database & Distributed Systems: Bridging the Divide

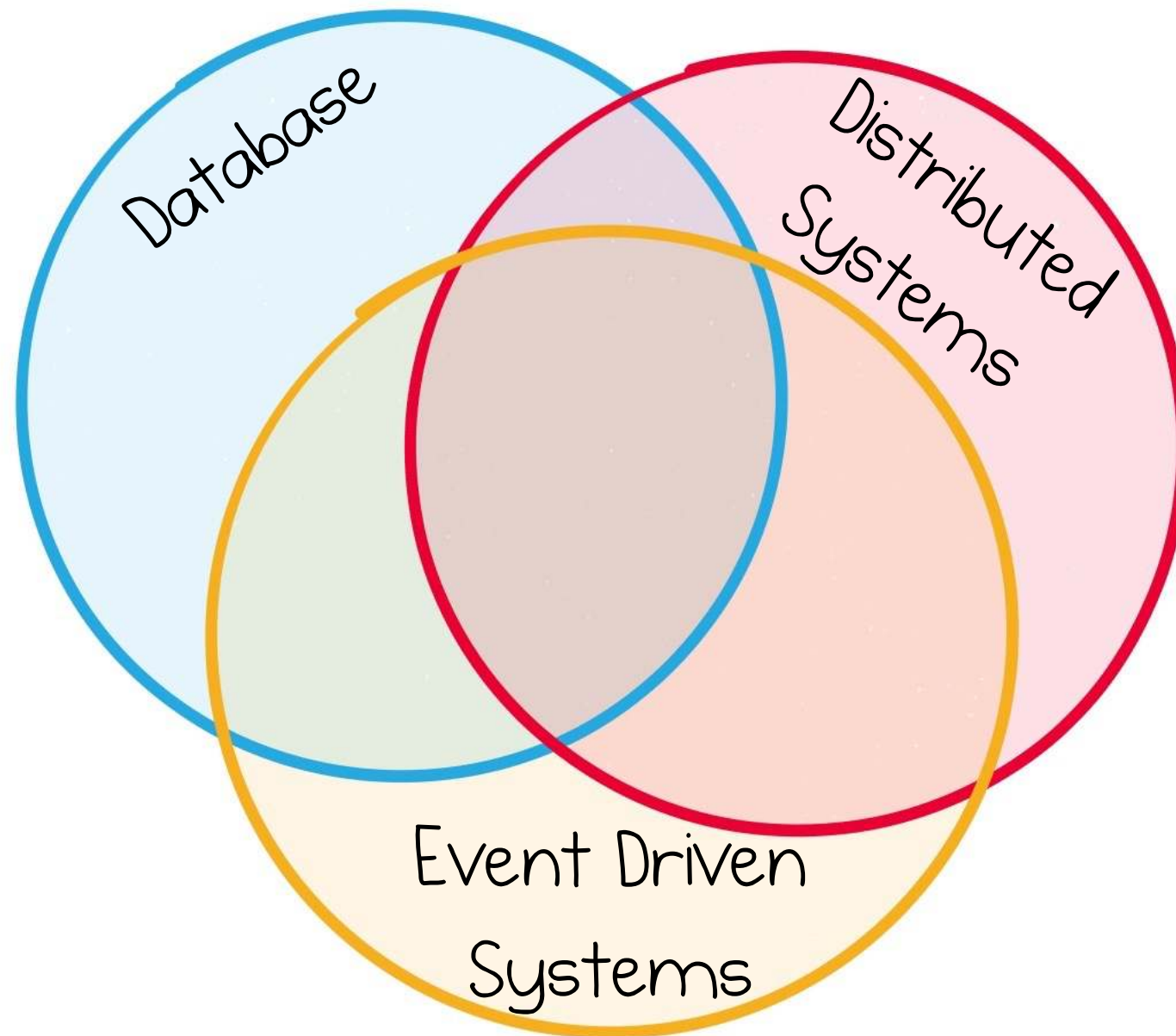
Ben Stopford @[confluentinc](https://twitter.com/confluentinc)

THIS IS ME

- ENGINEER
AT CONFLUENT
- Ex THOUGHTWORKS
+ UK FINANCE



Most stateful systems have to pull
from these three worlds



Today we have 2 goals

1. Understand Stateful Stream Processing (now & near future)
2. Case for SSP as a general framework for building data-centric systems.

Data systems come in different forms

- Database (OLTP)
- Analytics Database (OLAP/Hadoop)
- Messaging
- Distributed log
- Stream Processing
- Stateful Stream Processing

Database (OLTP)

Focuses on providing a consistent view that supports updates and queries on individual tuples.



Analytics Database (OLAP/Hadoop)

1. Focuses on aggregations via table scans.
2. Executes as distributed system



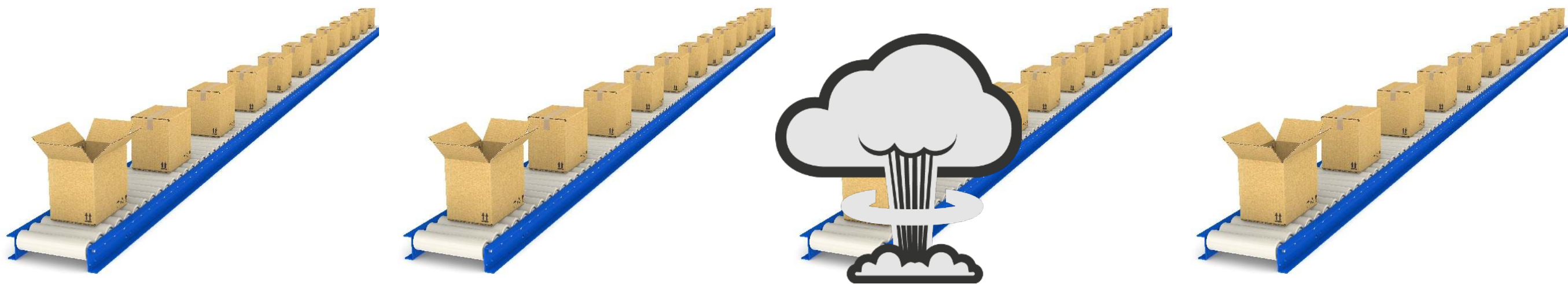
Messaging

Focuses on asynchronous information transfer with limited state



Distributed Log

1. Similar to messaging, but data can be retained
2. Executes as distributed system (scale + fault tolerance)



Stream Processing

Manipulate concurrent streams of events

Comes from CEP background (ephemeral)

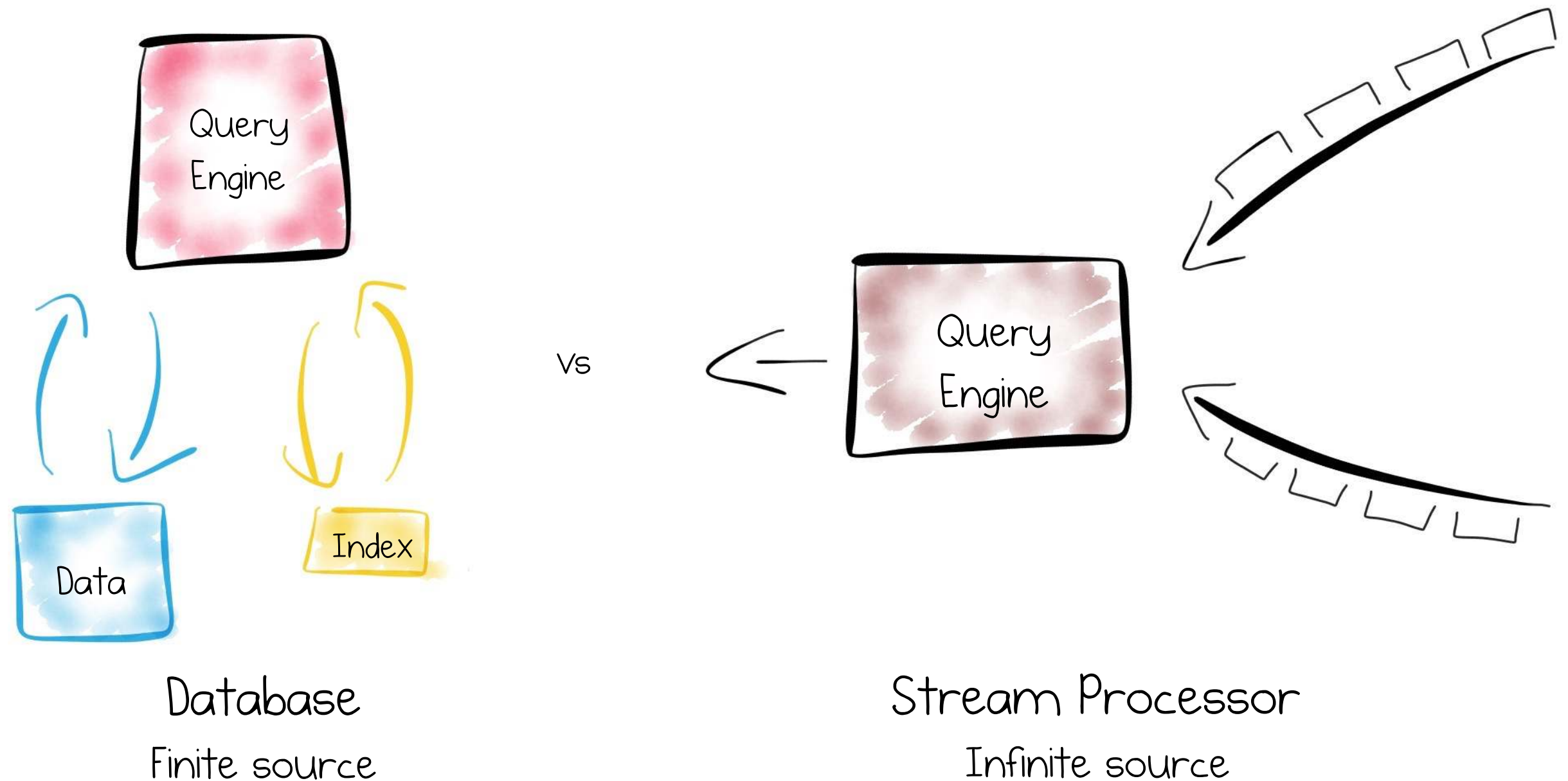


Stateful Stream Processing

Moves stream processing to be a more general framework for building data-centric systems.



What is stream processing?



Infinite streams need windows



How many items will we bring into the machine at one time?

Windows bound a computation



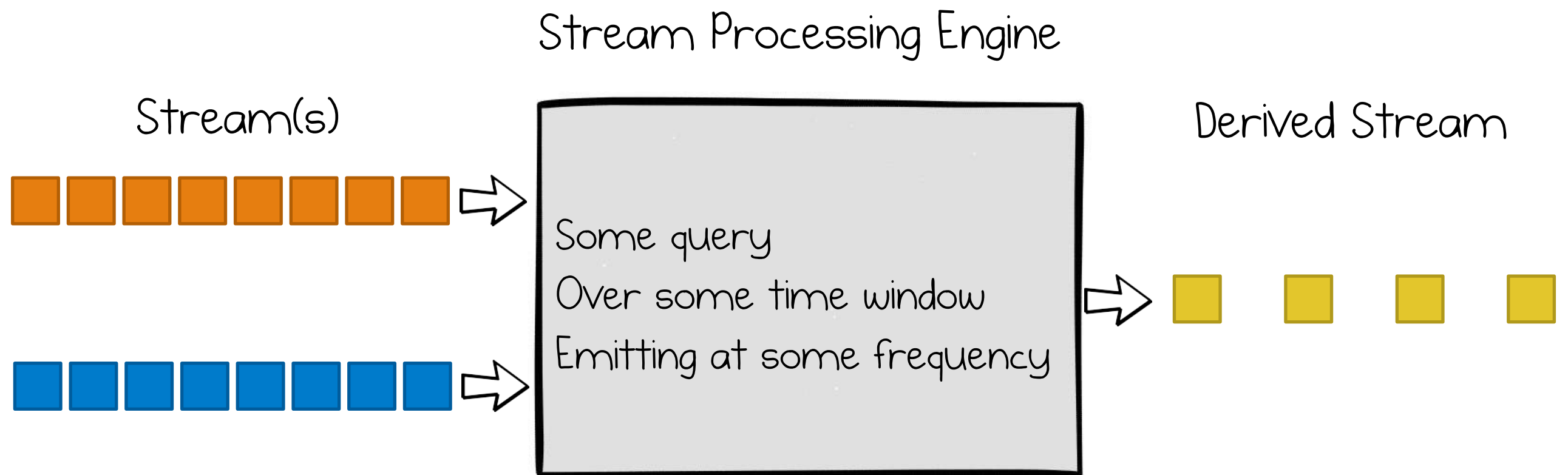
How many items will we bring into the machine at one time?

Buffering allows us to handle late events

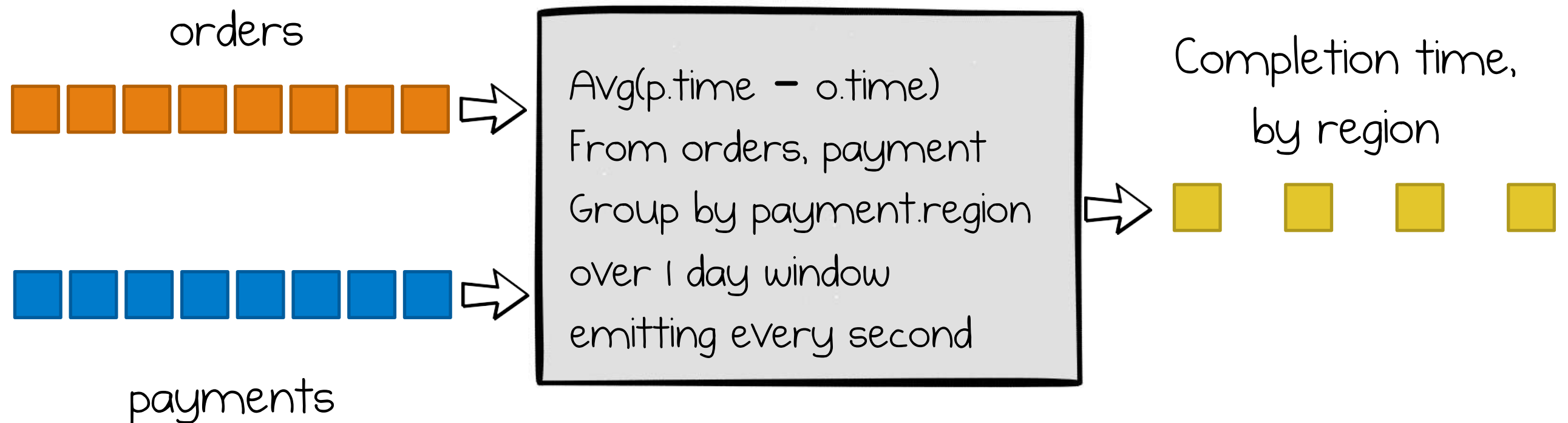


How many items will we bring into the machine at one time?

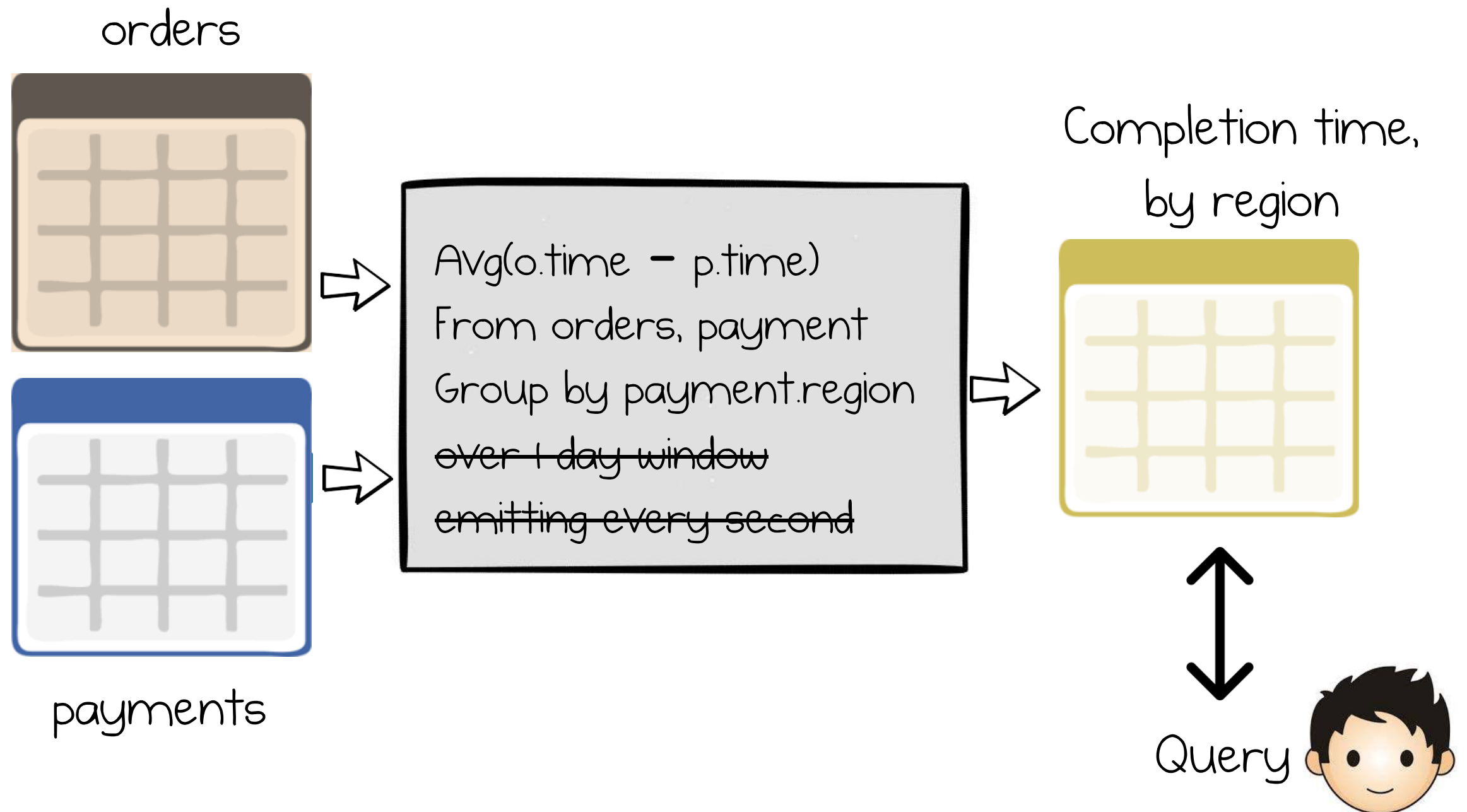
Continually executing query



Stream Processing



Materialised View (DB)



Stateful Stream Processing

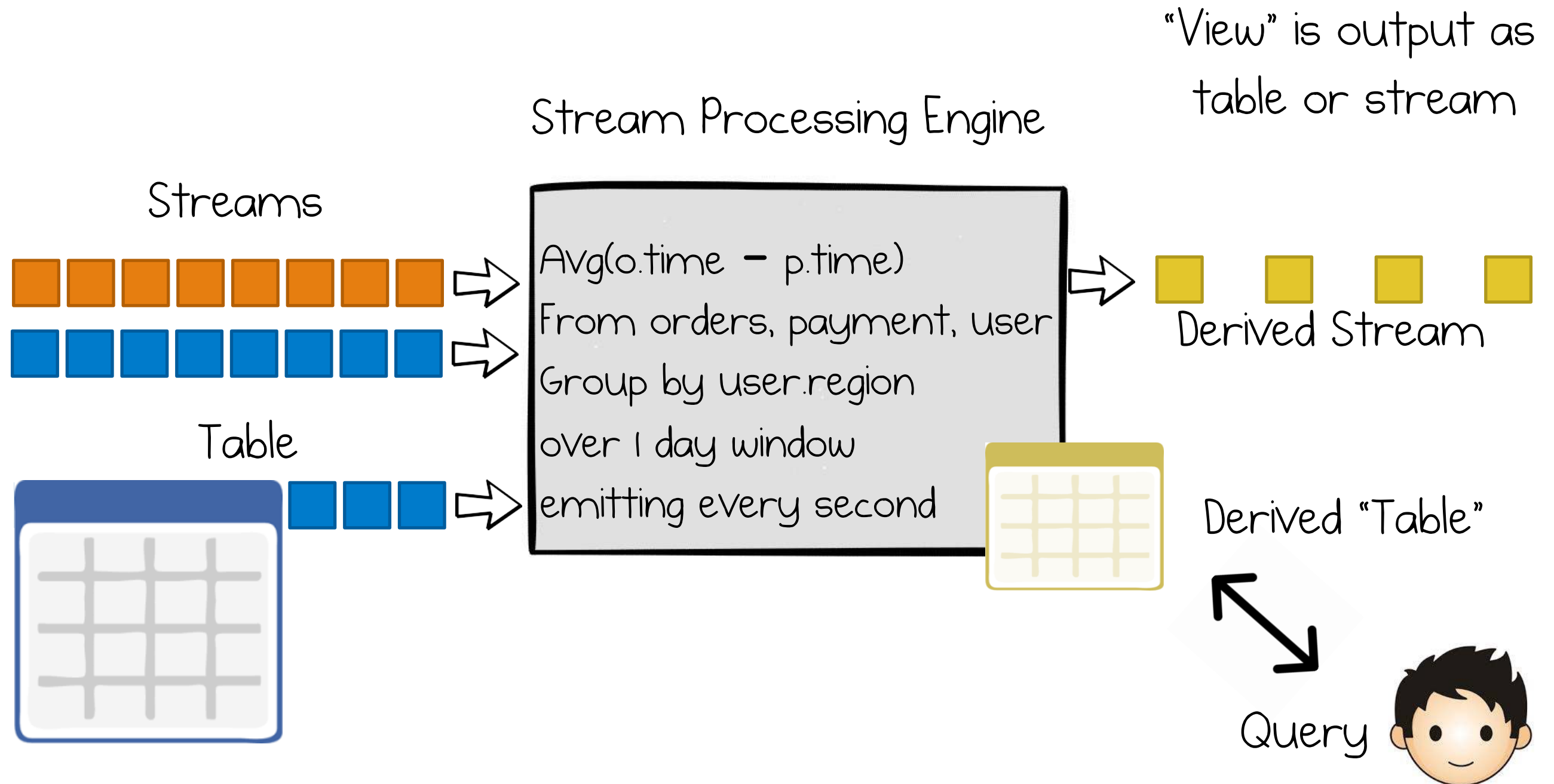


Table == Stream + Window₀ⁿ

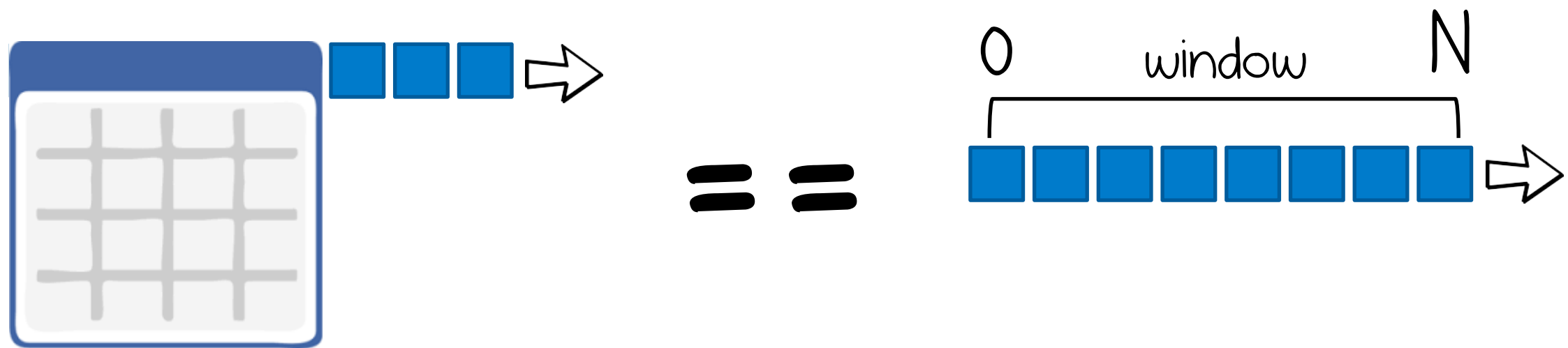
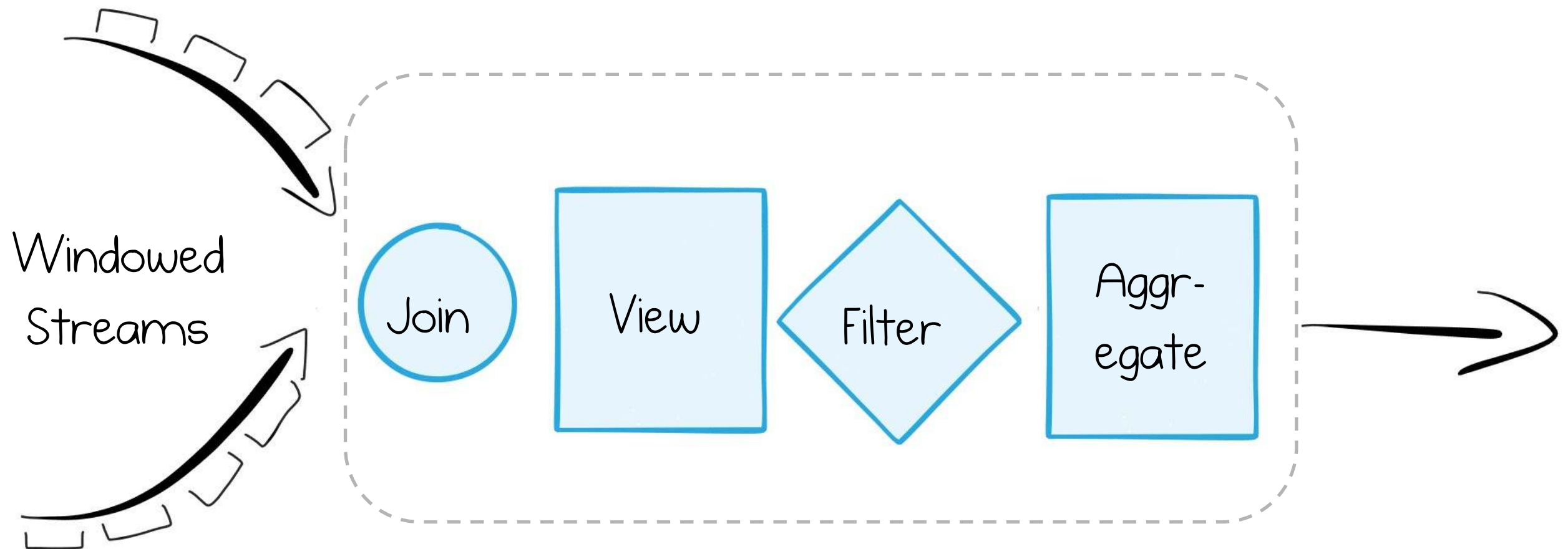


Table is a stream with an infinite window (i.e. buffer from 0 -> now)

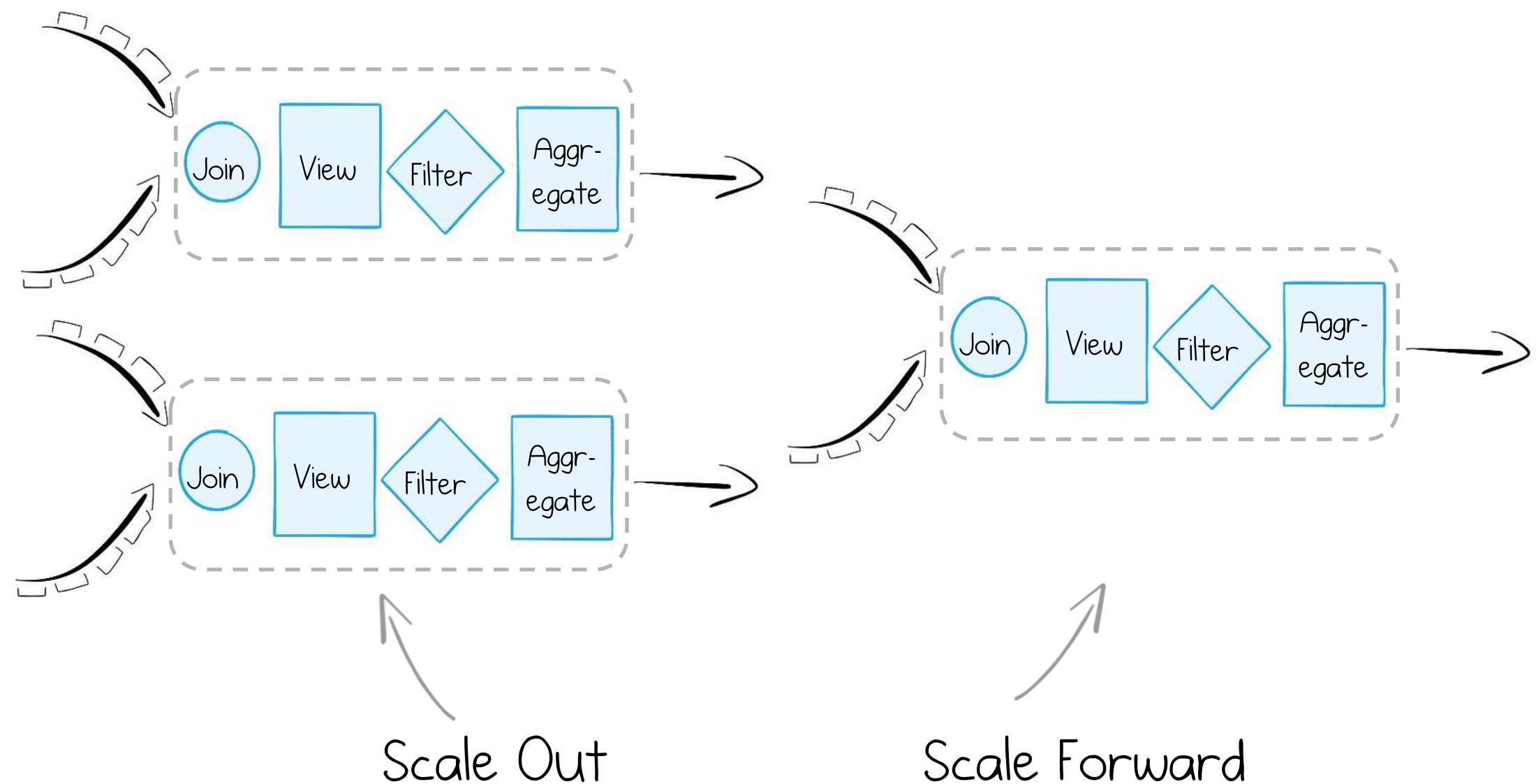
SSP is about creating
materialised views.

Materialised as a table, or
materialised as a stream

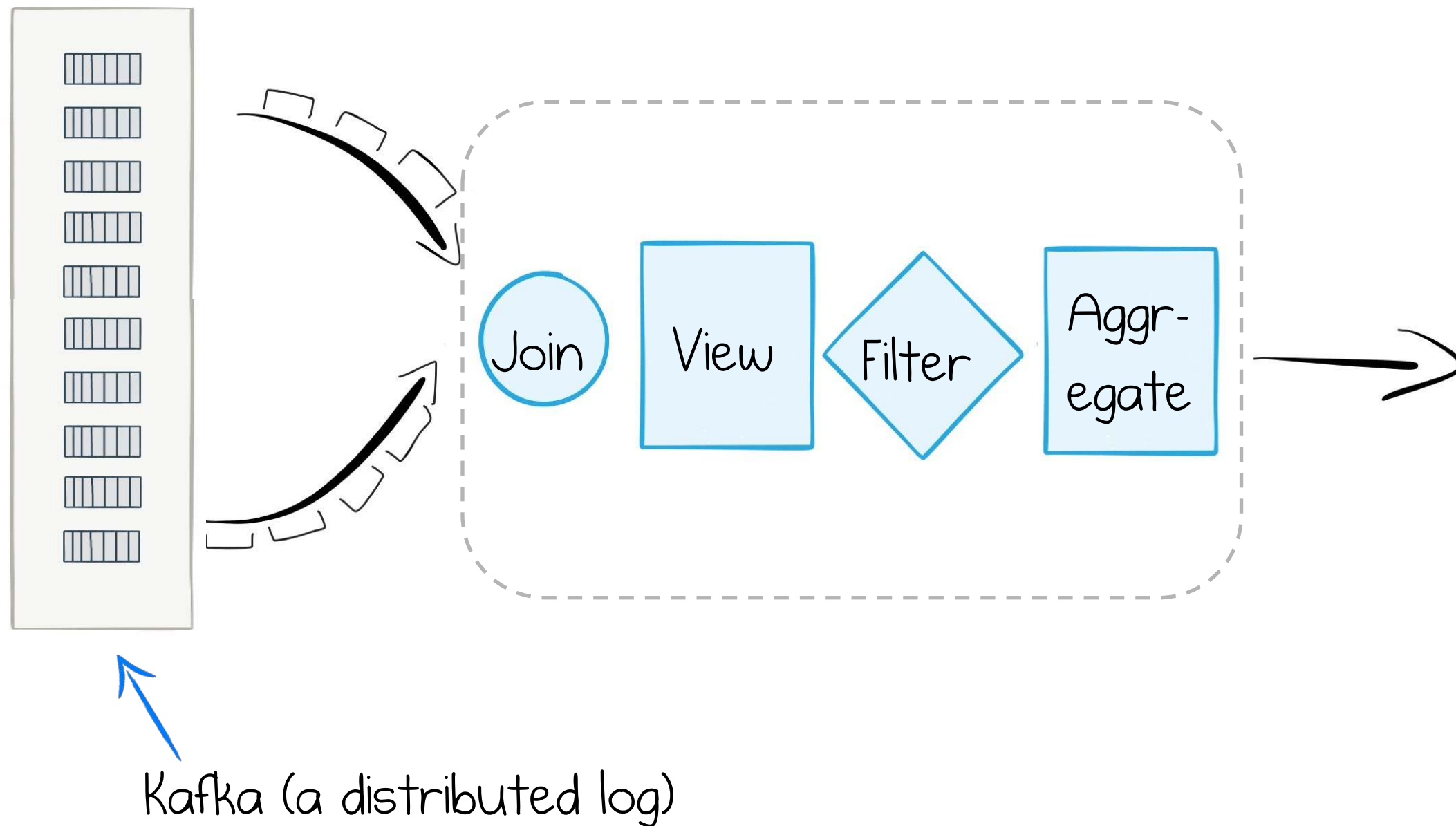
Features: similar to database query engine



Can distribute over many machines in two dimensions

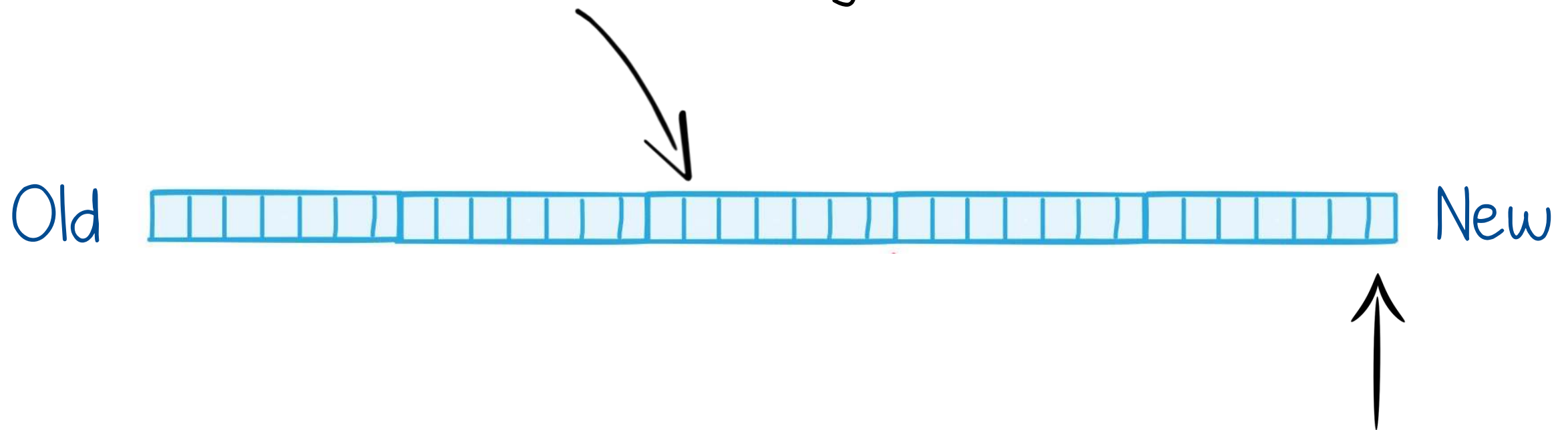


Stateful Stream Processing engines typically use Kafka (a distributed commit log)



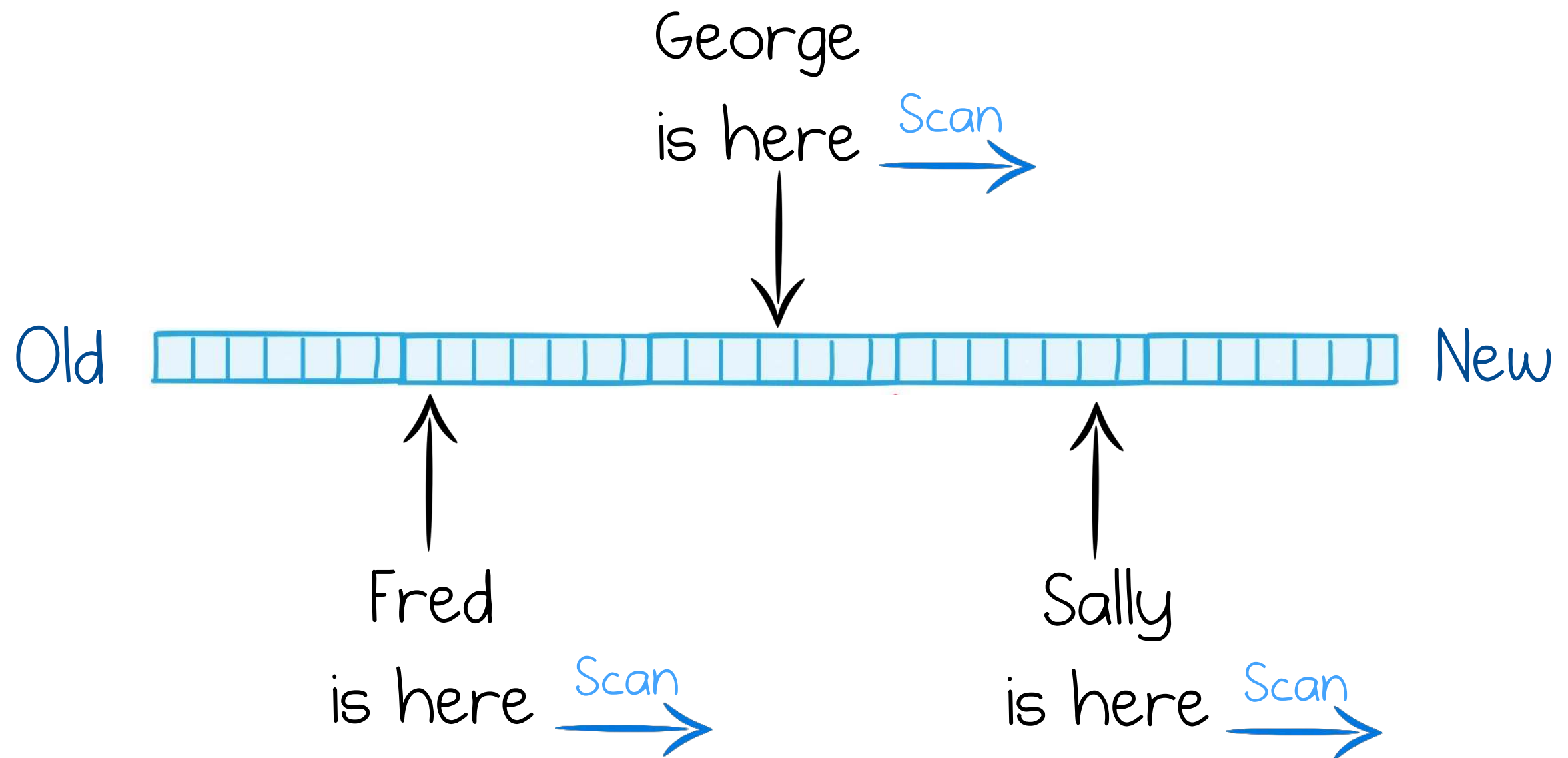
A log is very simple idea

Just think of the log as a file

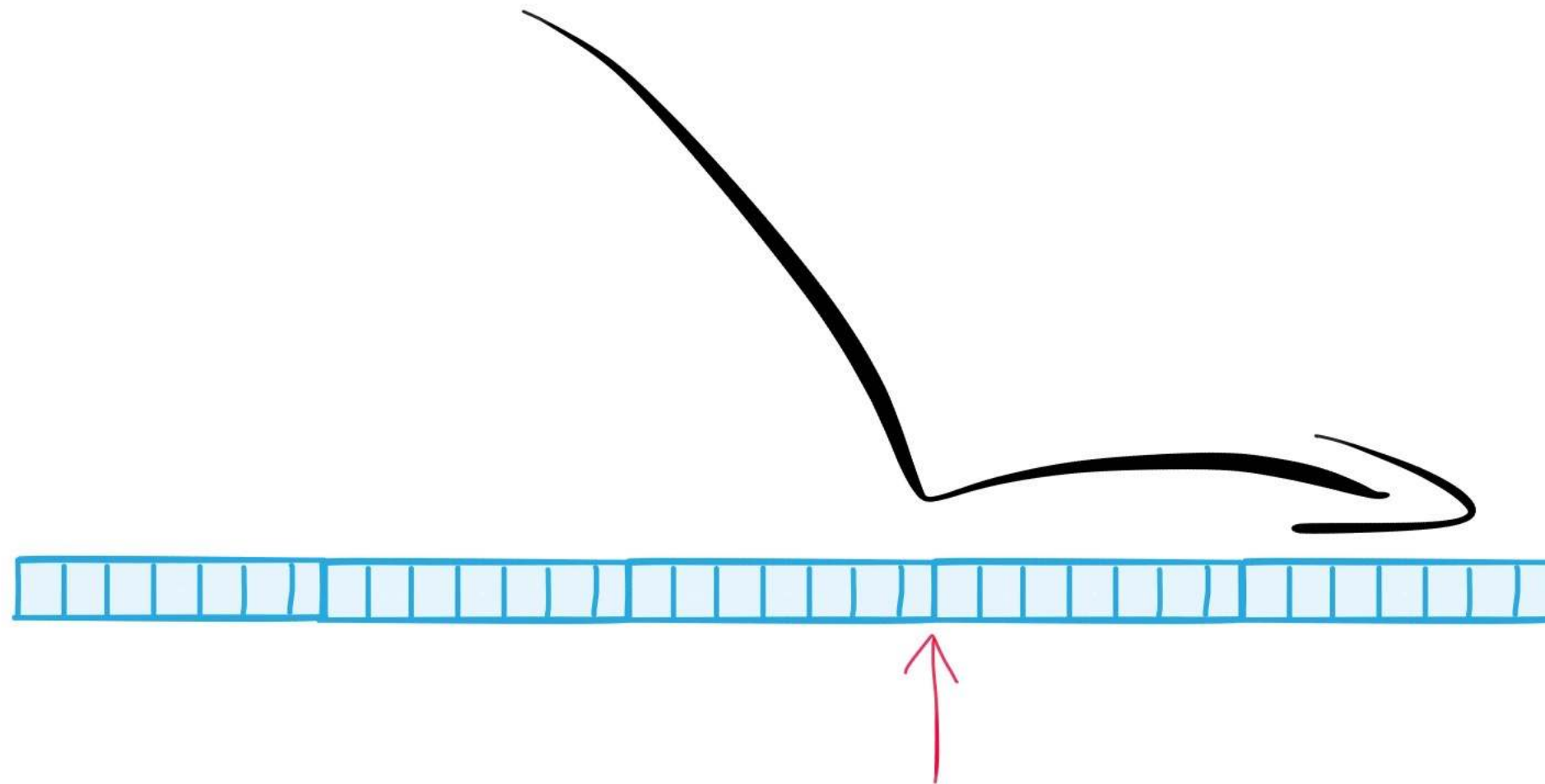


Messages are added at the end of the log

Readers have a position & scan



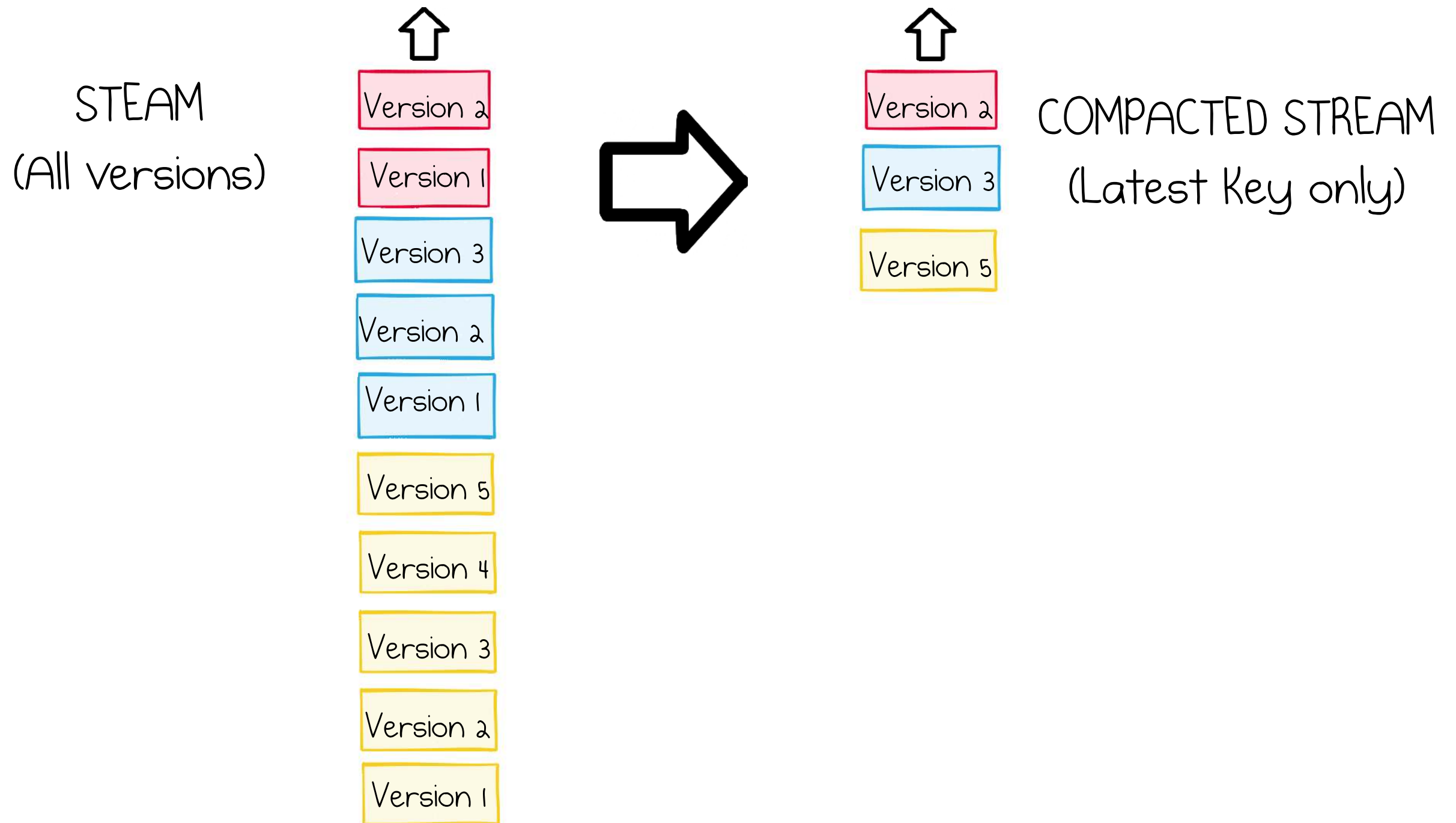
Can "Rewind & Replay" the log



Rewind & Replay

Compacted Log

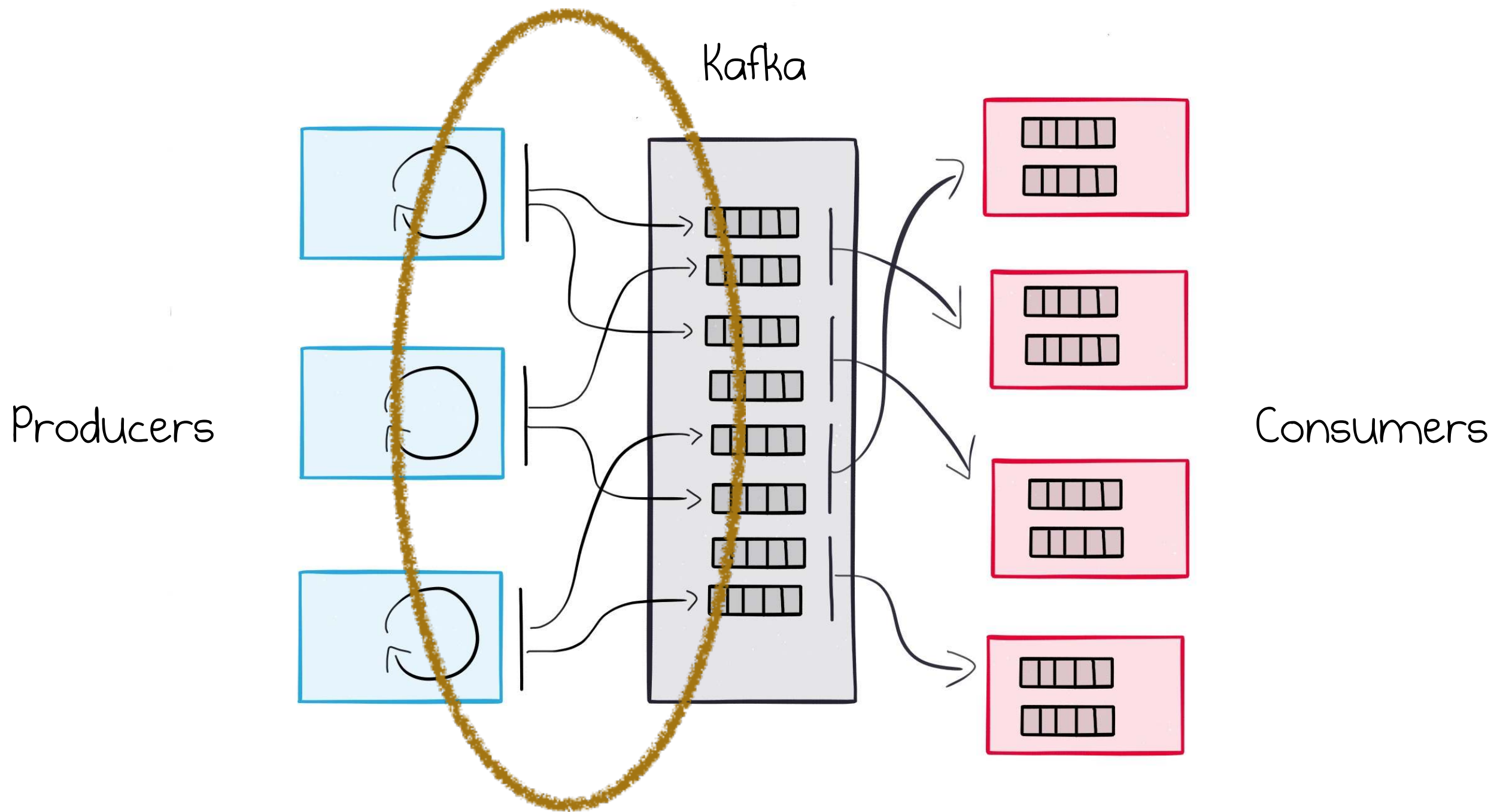
(Tabular View)



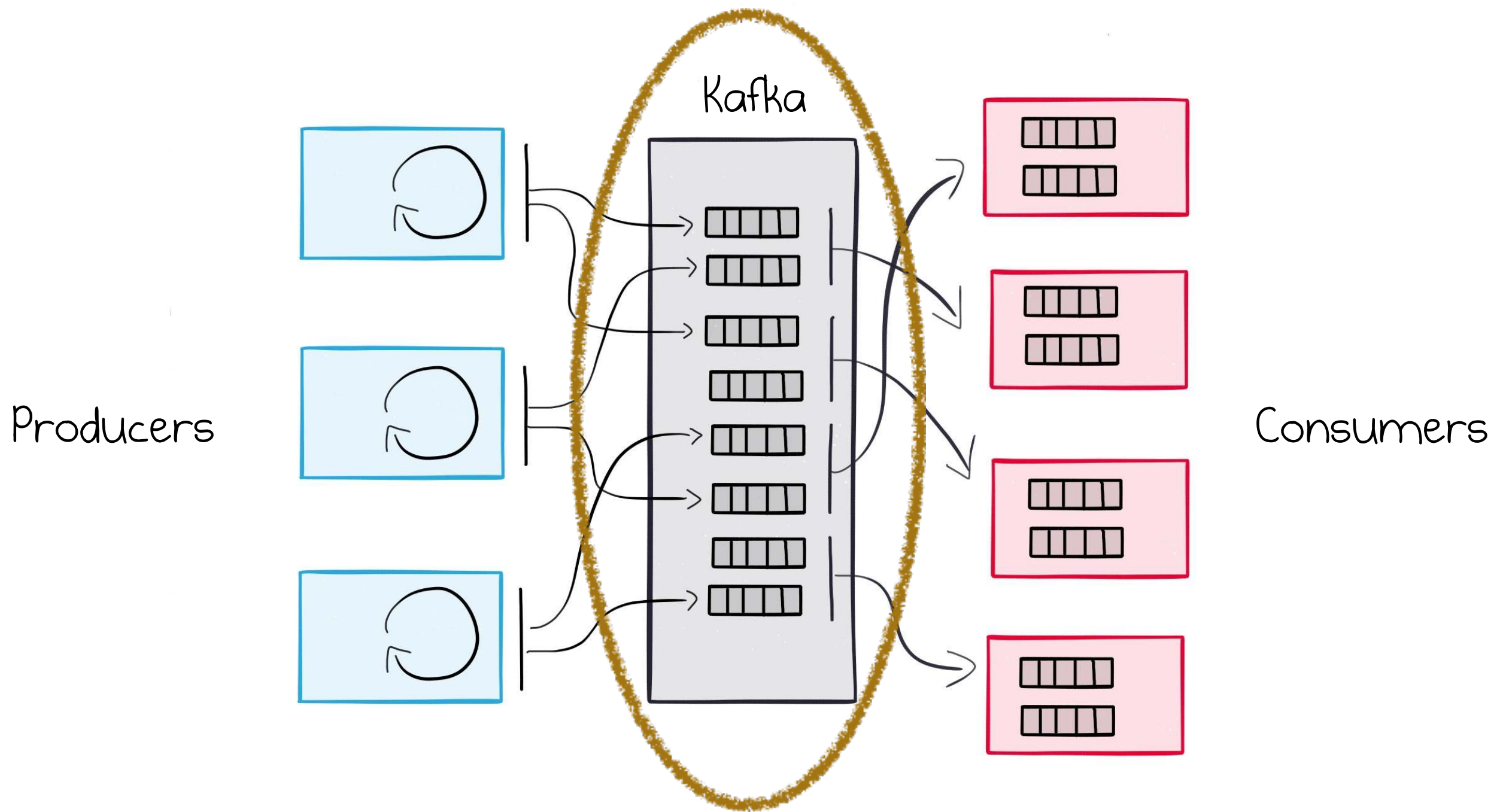
The log is a Distributed System

For scalability and fault tolerance

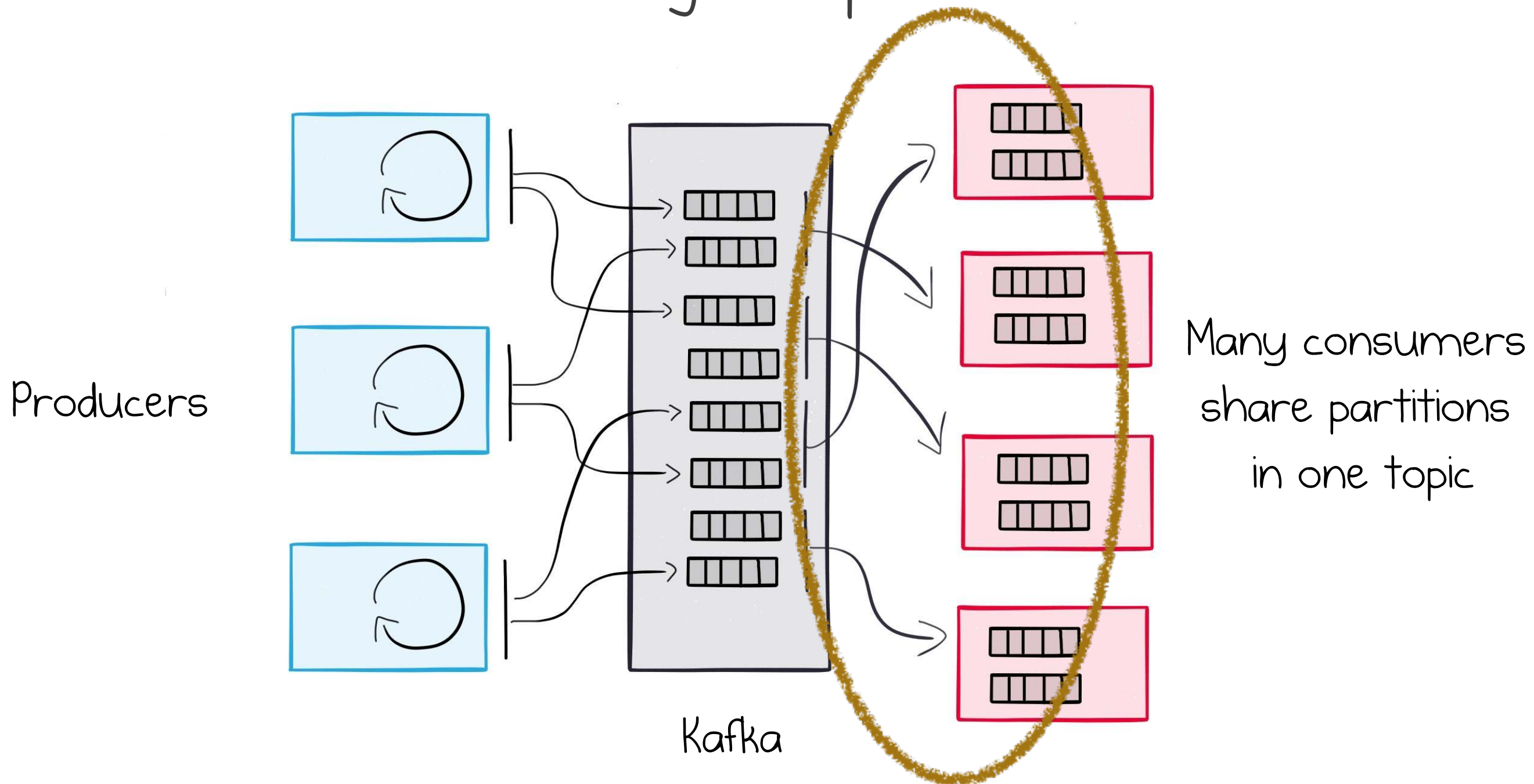
Shard on the way in



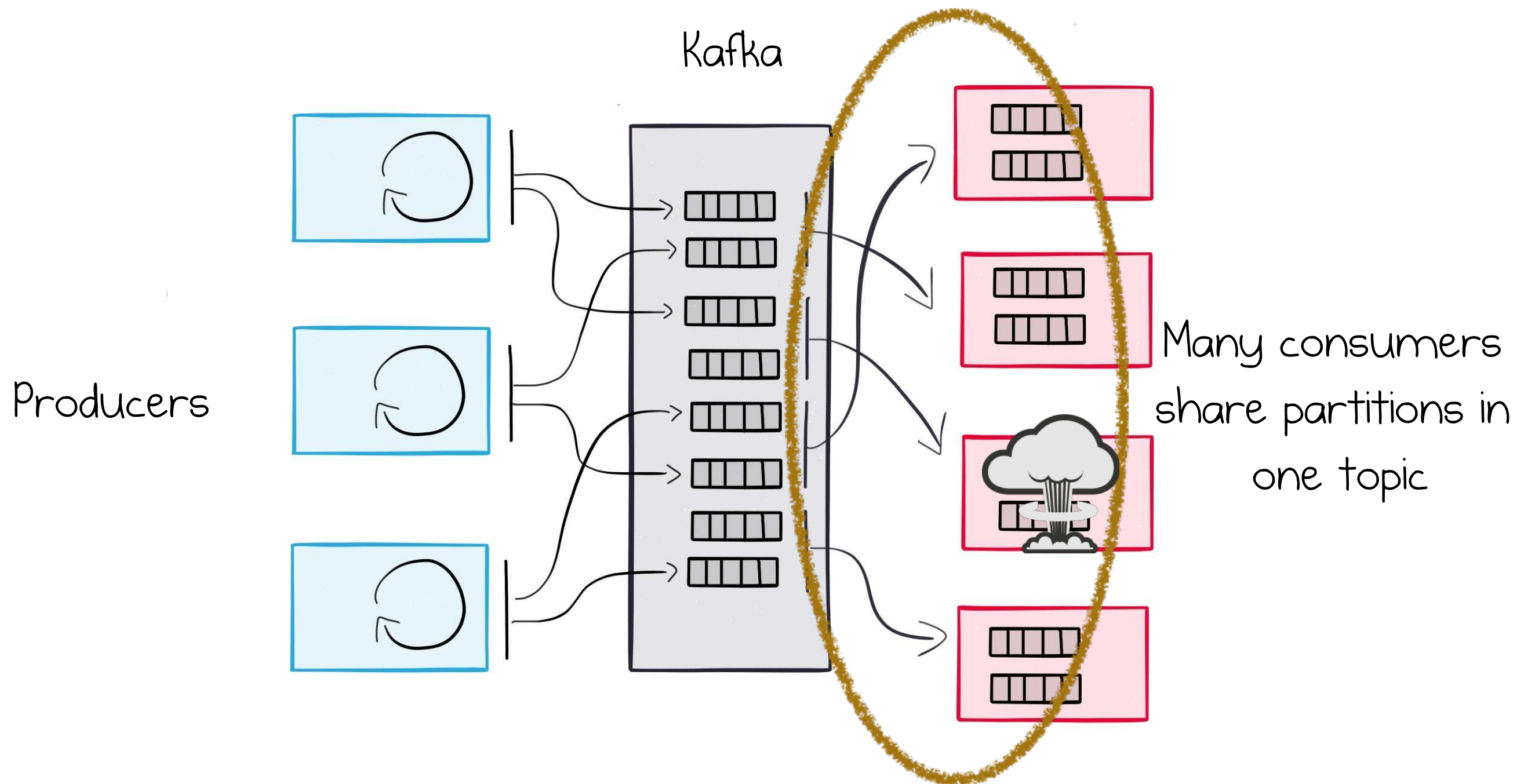
Each shard is a queue



Consumers share consumption of a single topic



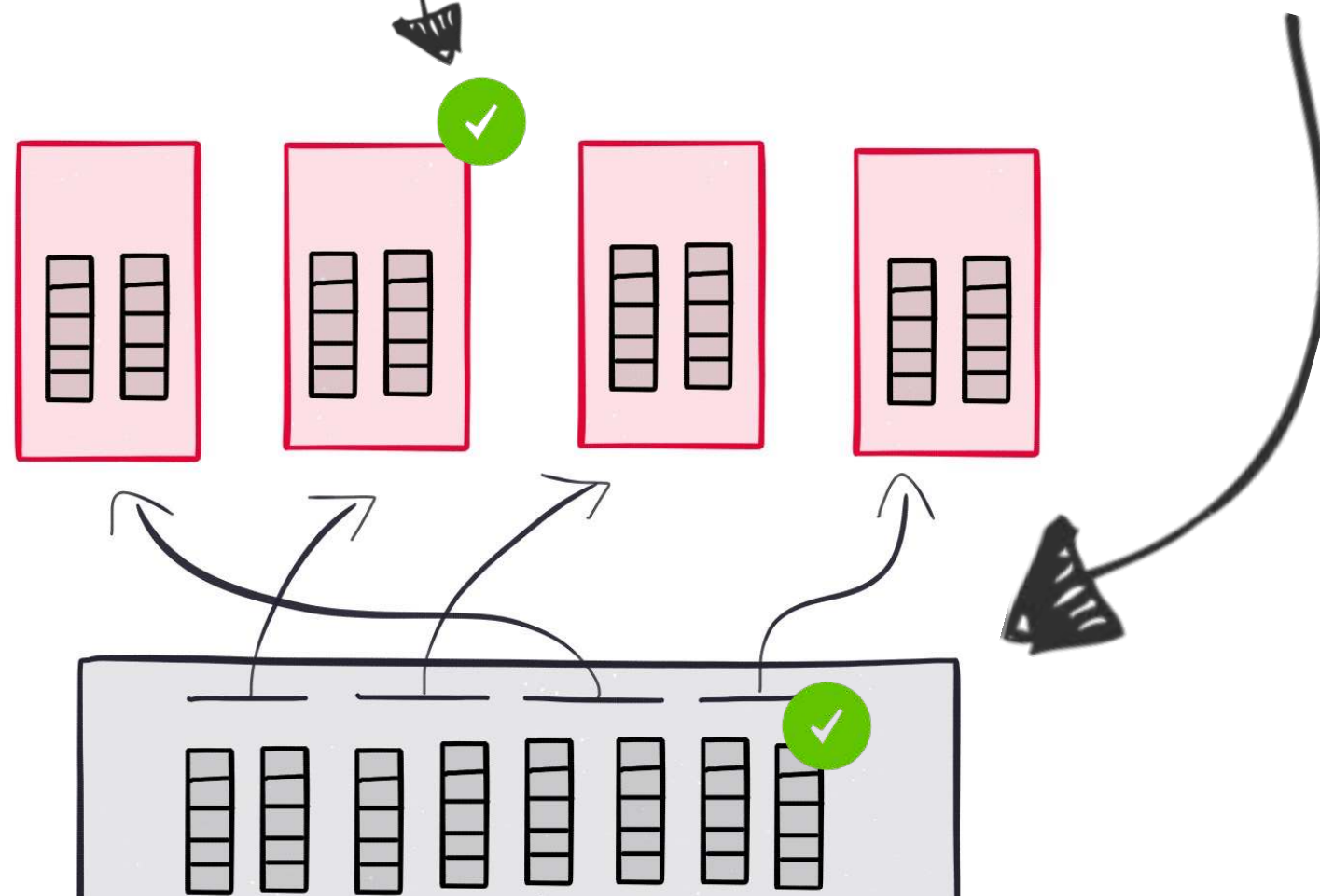
The Log reassigns data on failure



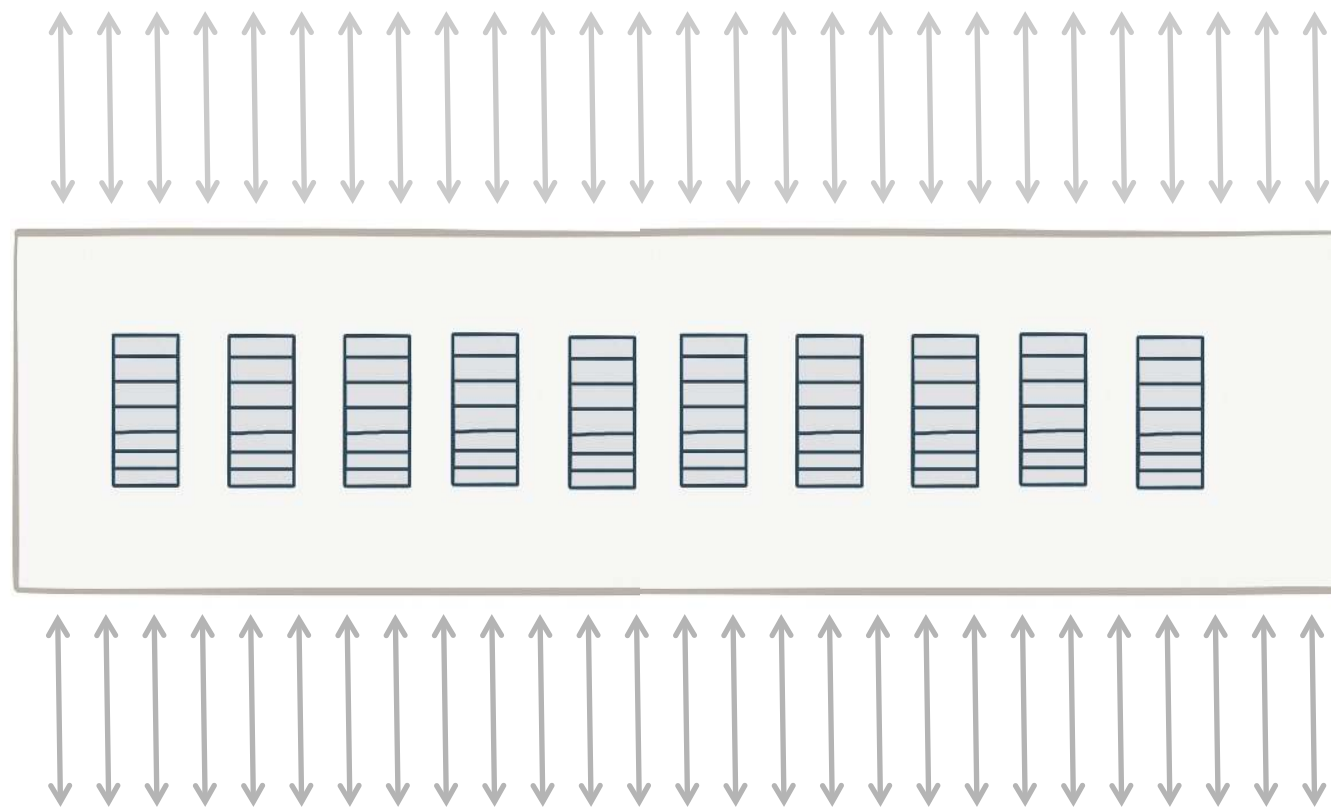
Kafka supplies two levels of leader election

Consumers in Kafka have an elected leader

Replicas in Kafka have an elected leader



The log is important for SSP



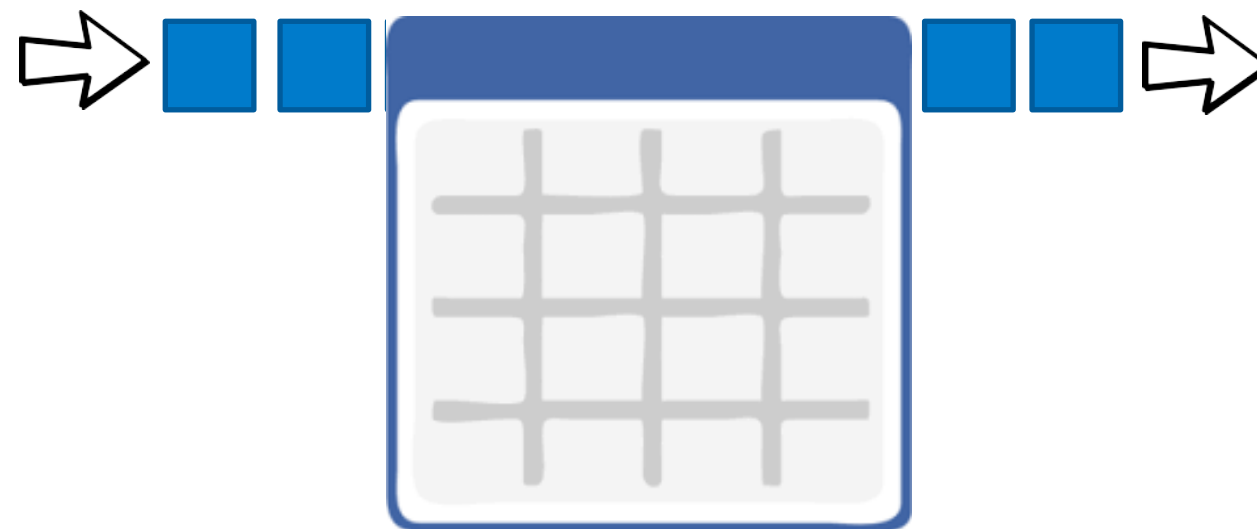
Maintains History: Acts like a “push based” distributed file system

The log is important: Two Primitives

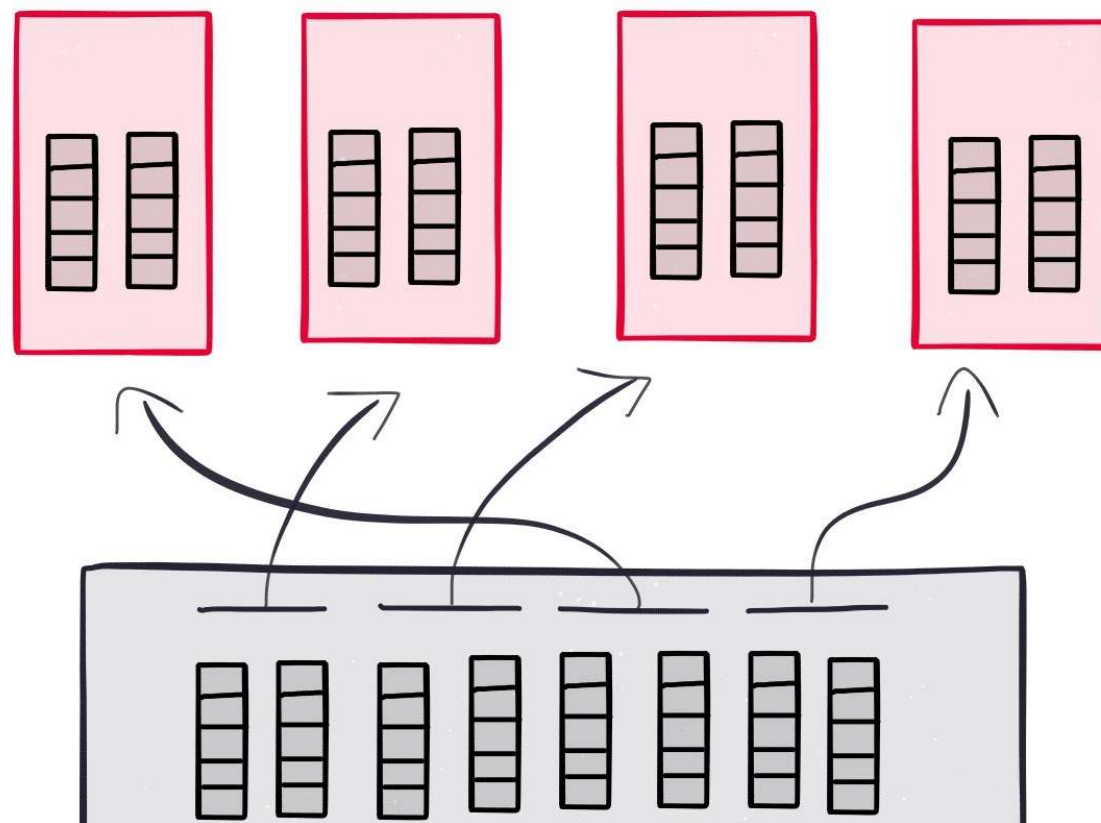
Stream



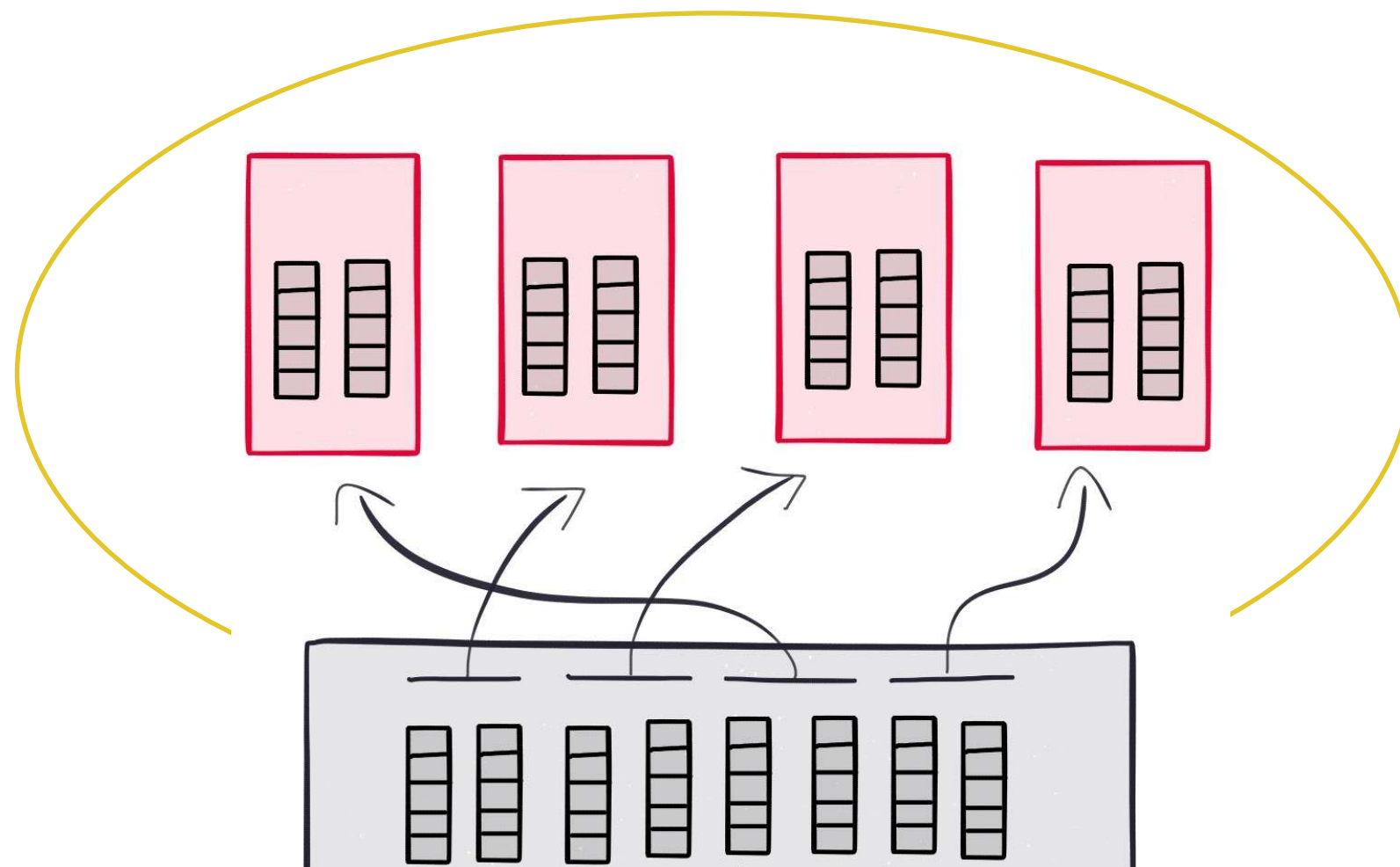
Compacted Stream ('table')



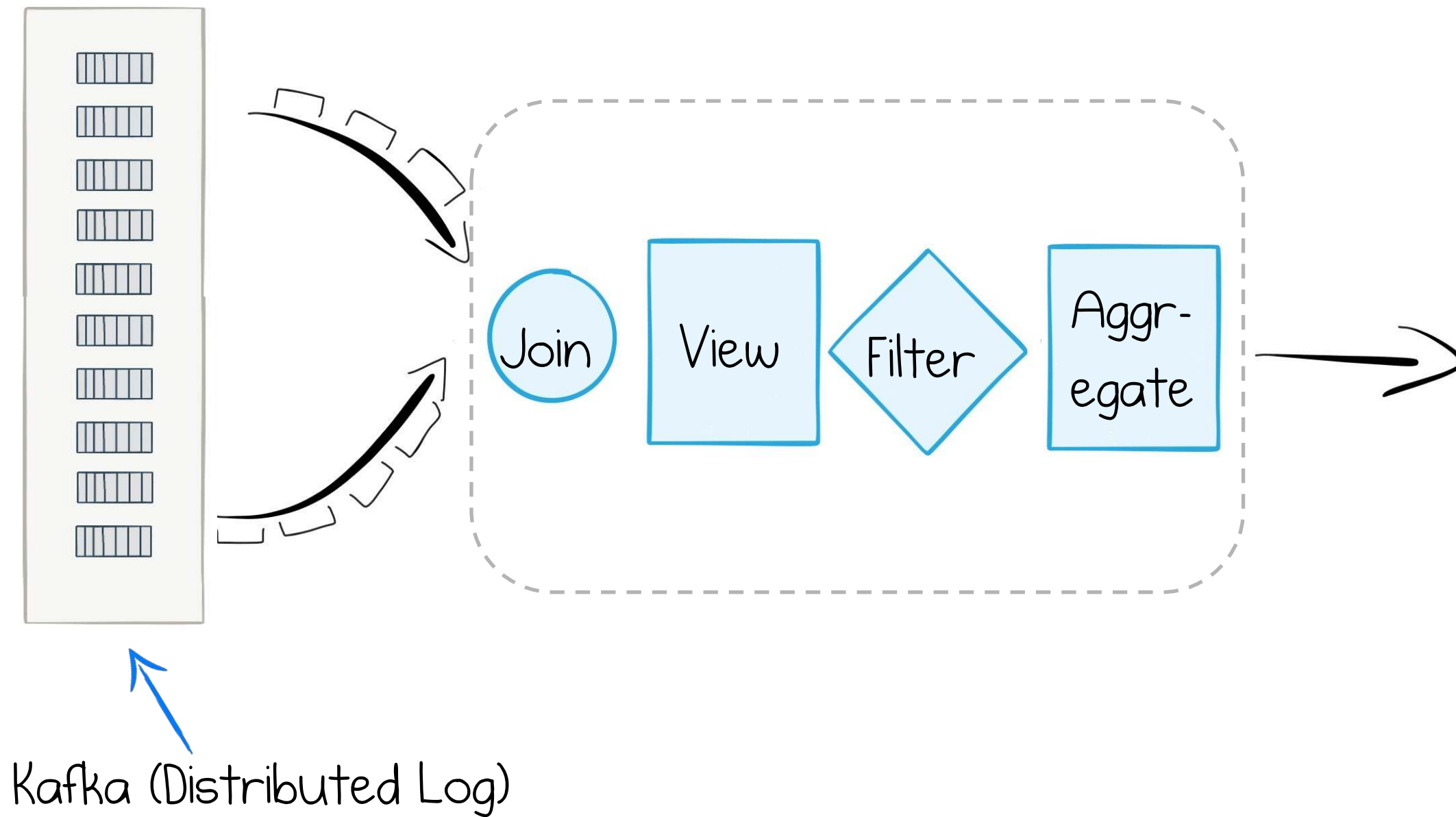
The Log is, to a streaming engine, what HDFS is to Hadoop



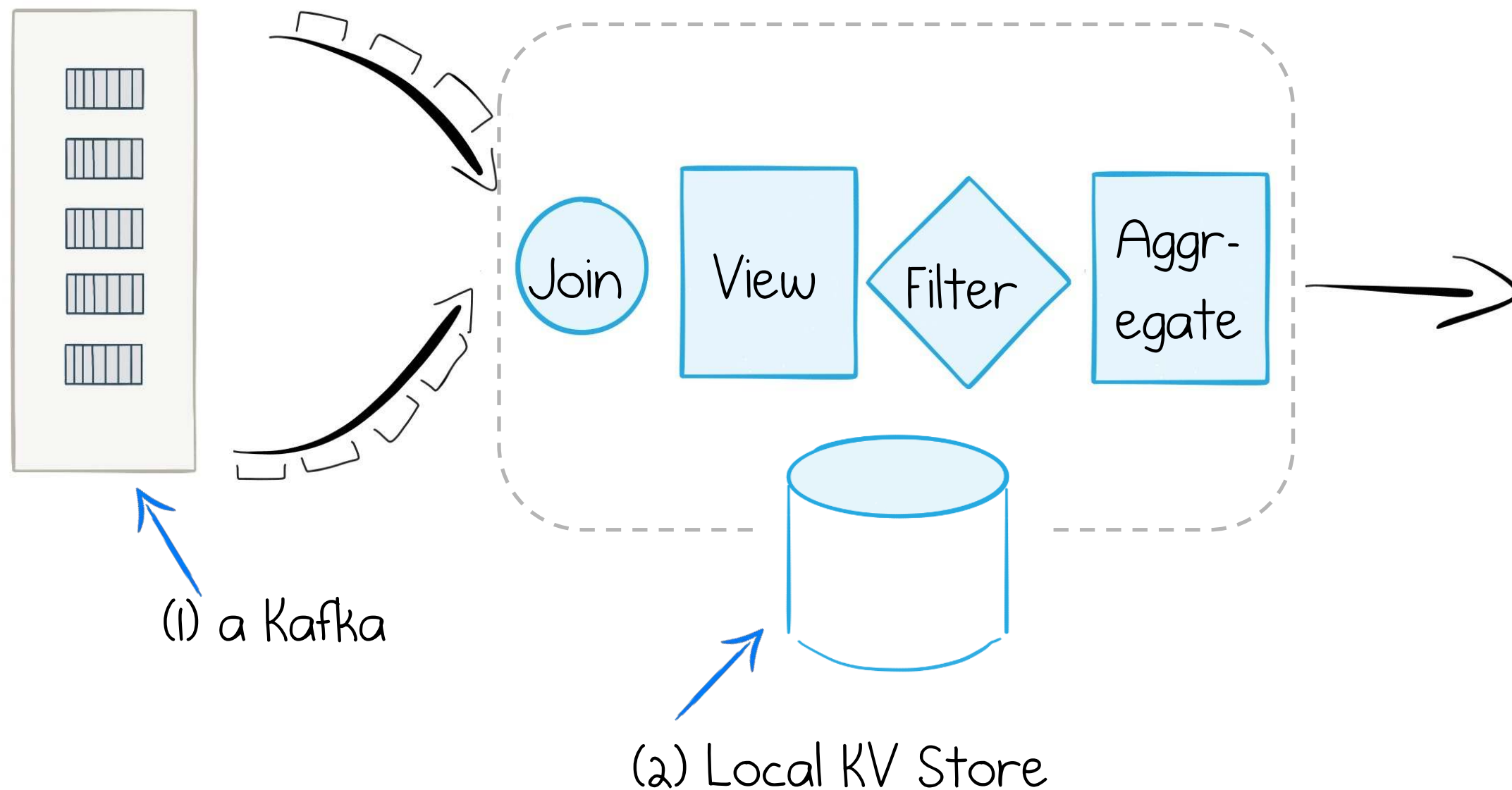
But it's a bit more than a HDFS replacement: Processors inherit the idea of "membership" from the log



So stateful Stream Processors use the Log

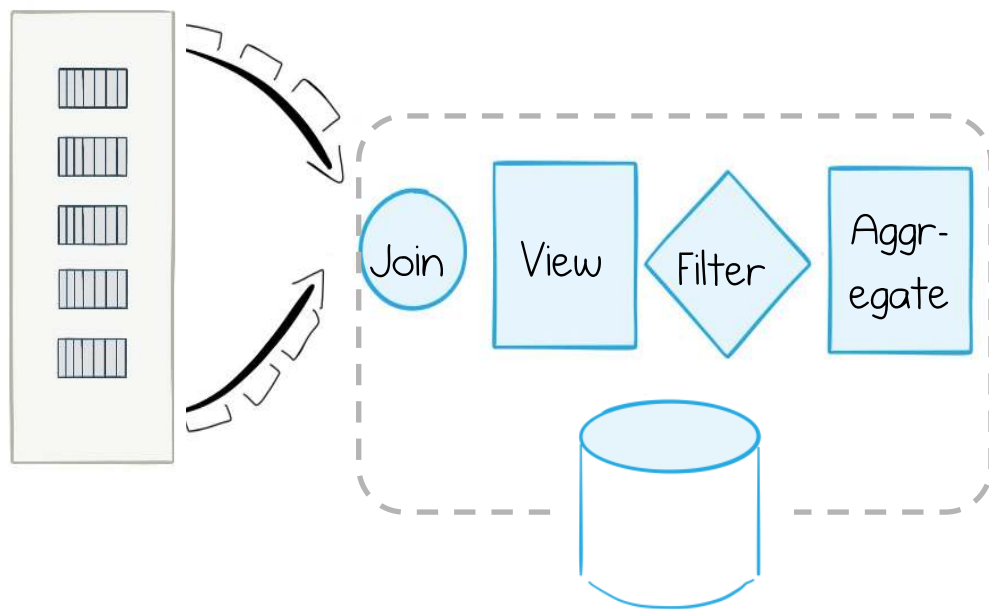


They also use local storage



Local KV store has a few uses

Streams just cache recent messages to help with joins

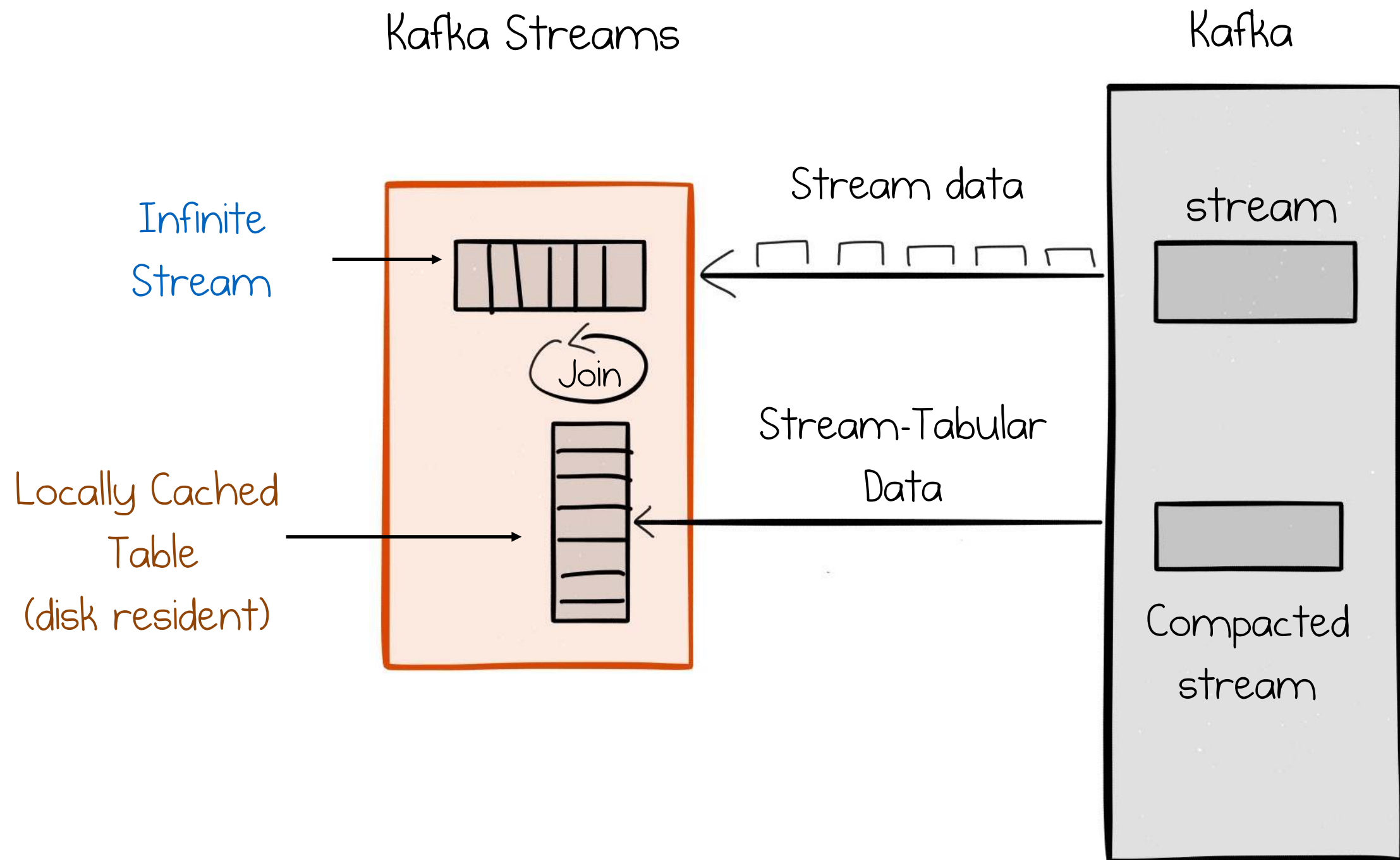


- (1) It caches streams on disk
- (2) It caches “tables” on disk

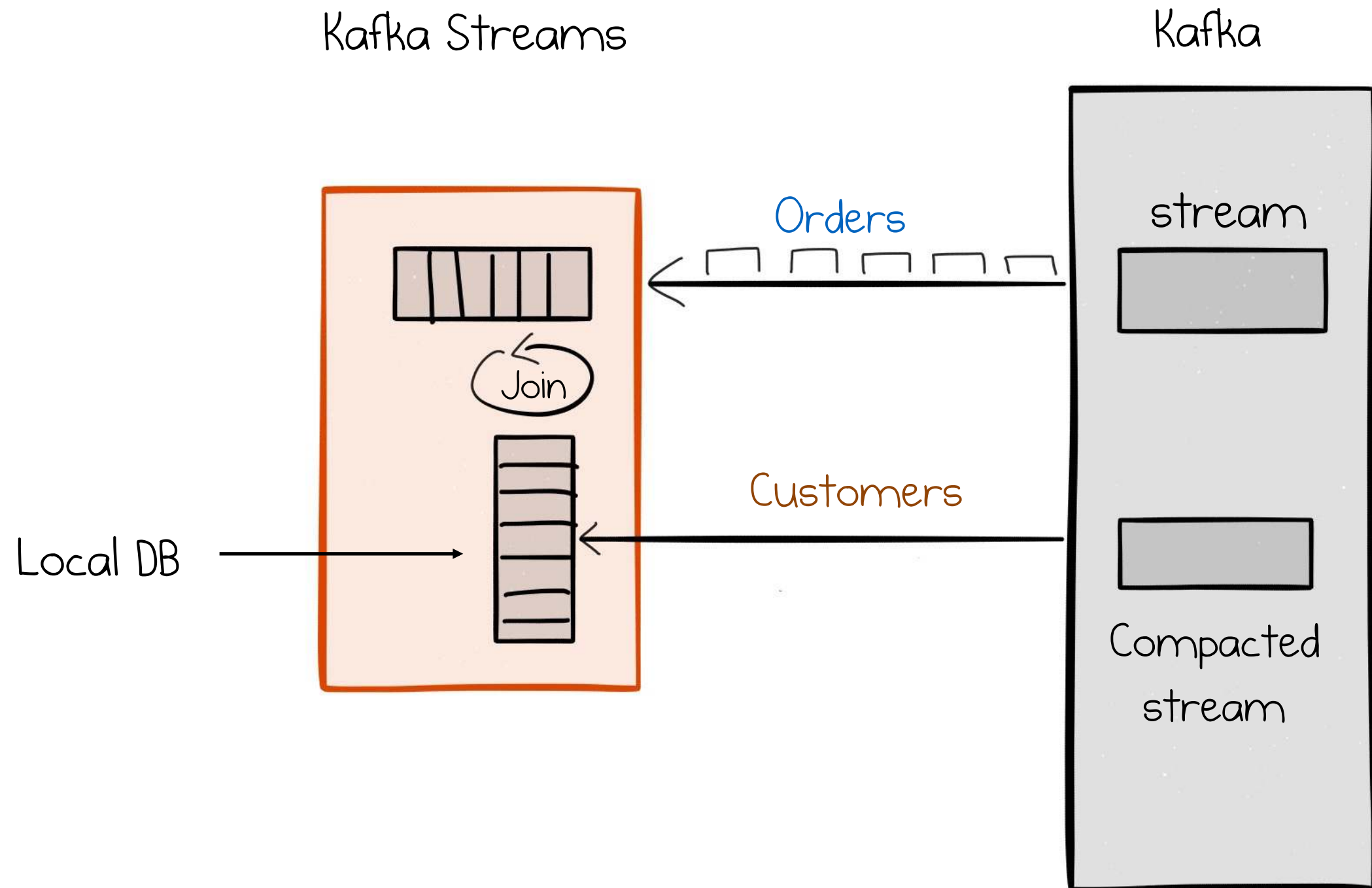
Tables are fully
“realised” locally

This makes join operations fast as they’re entirely local

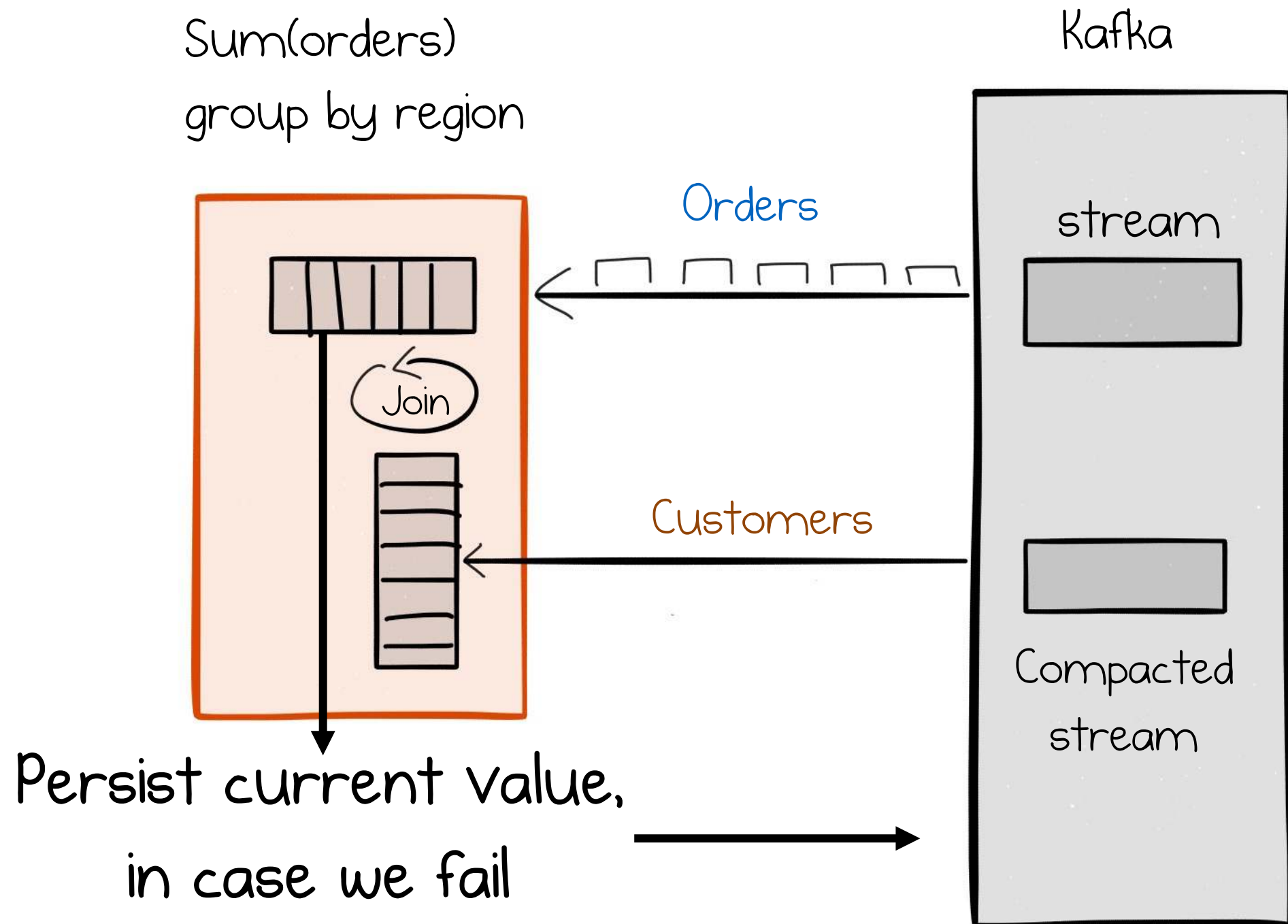
Stateful Stream Processing



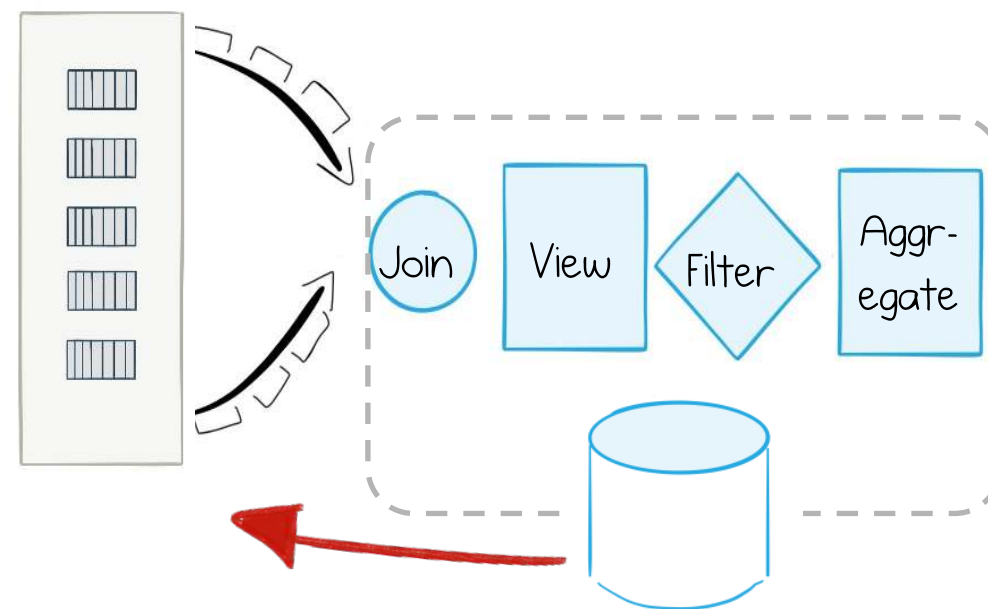
e.g. Useful for Enrichment



Aggregates need intermediary state

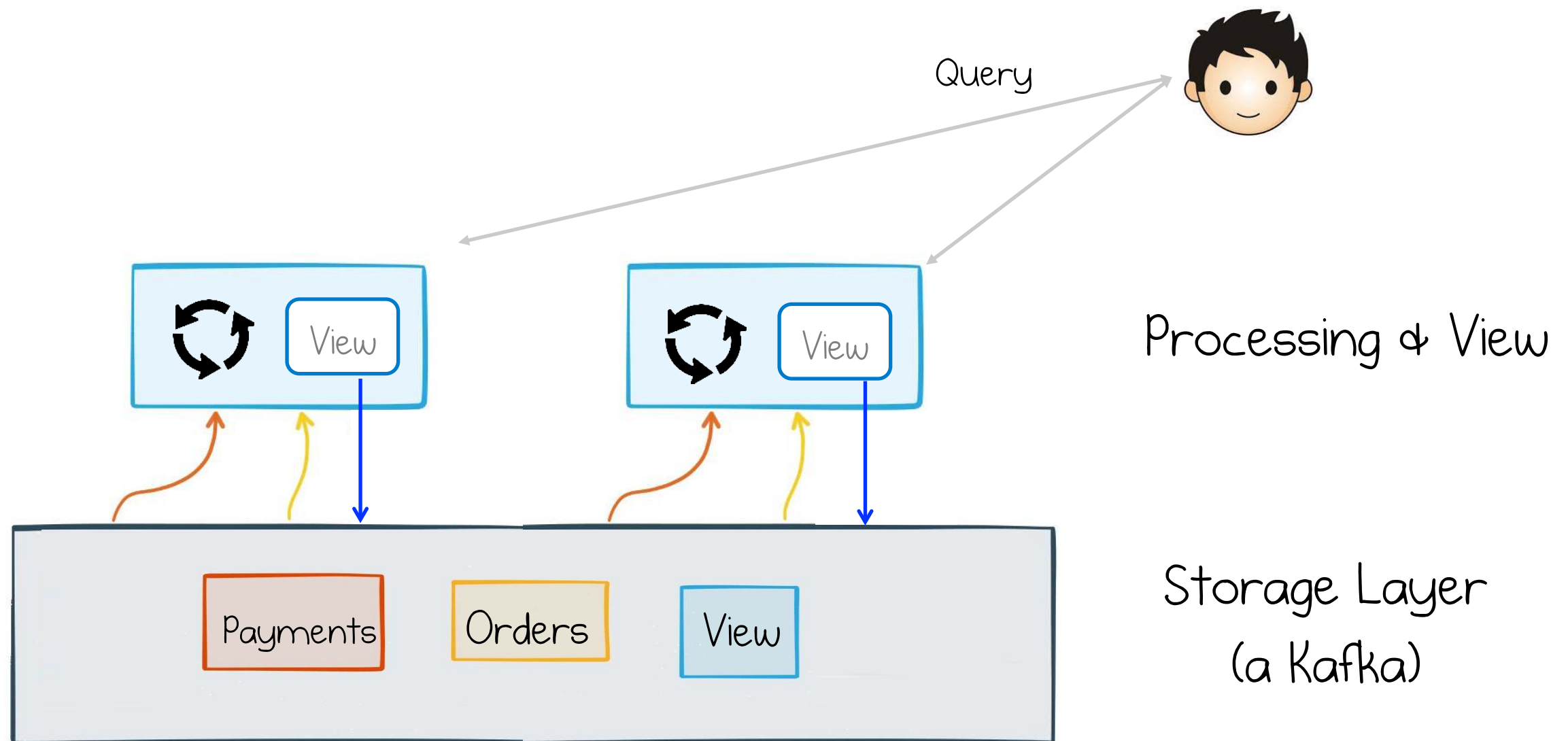


State store inherits durability from the log

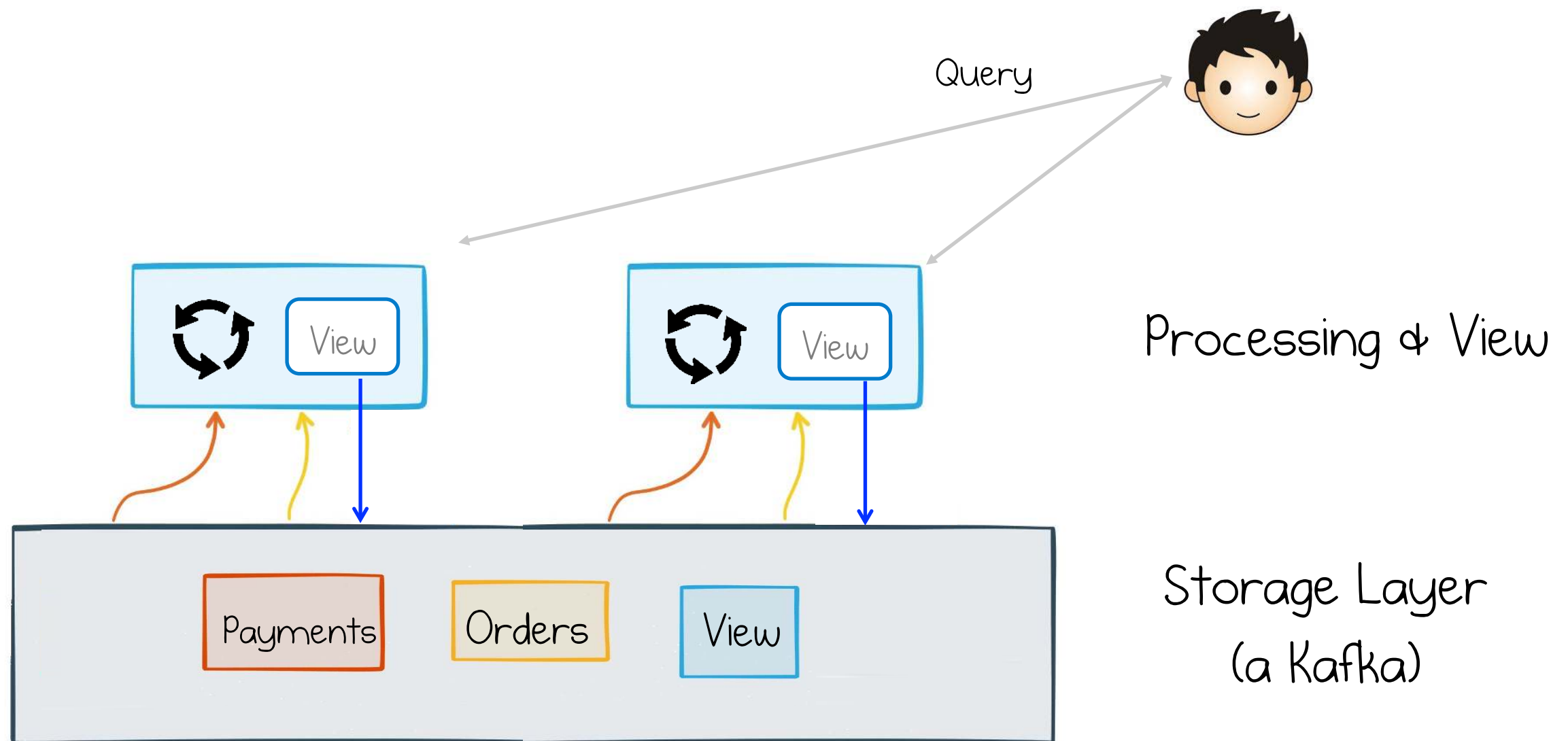


State store flushes
back to the log

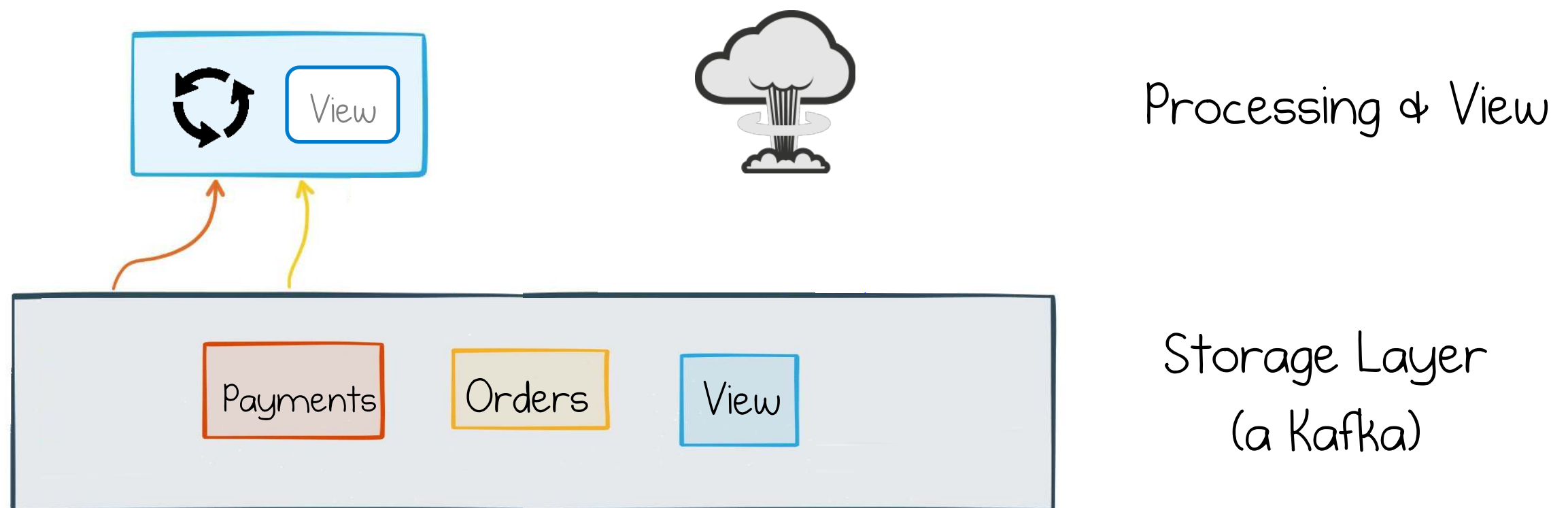
Separate Data, Processing & View



You can query the views from anywhere

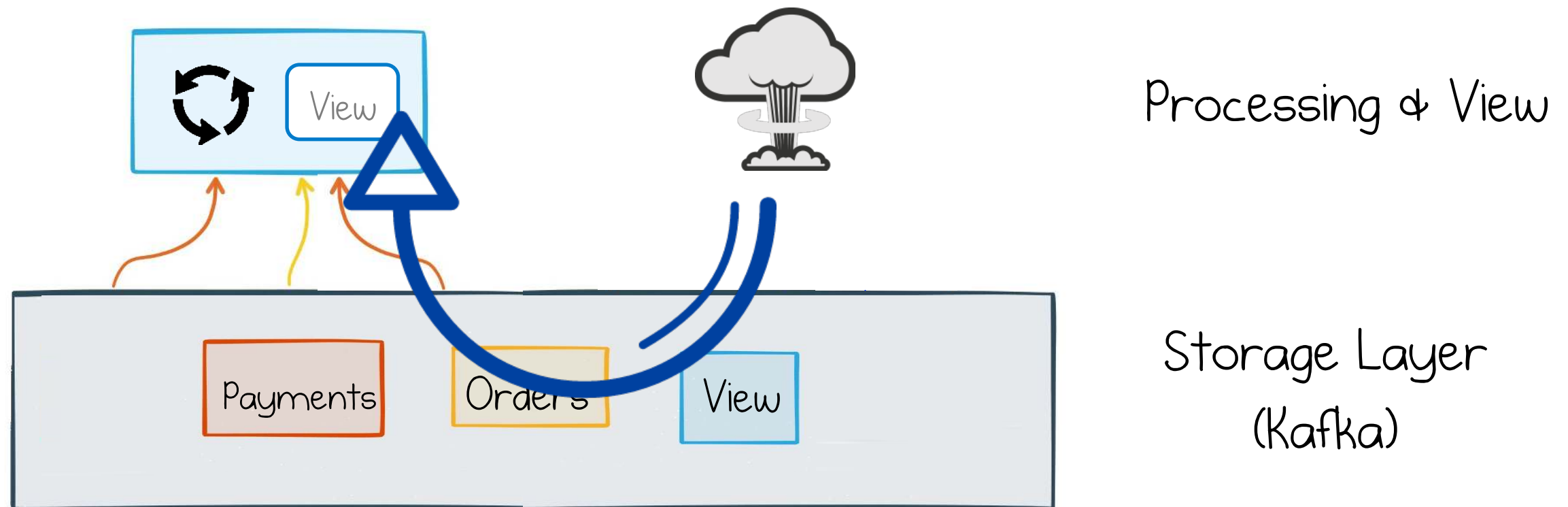


So what happens on failure?



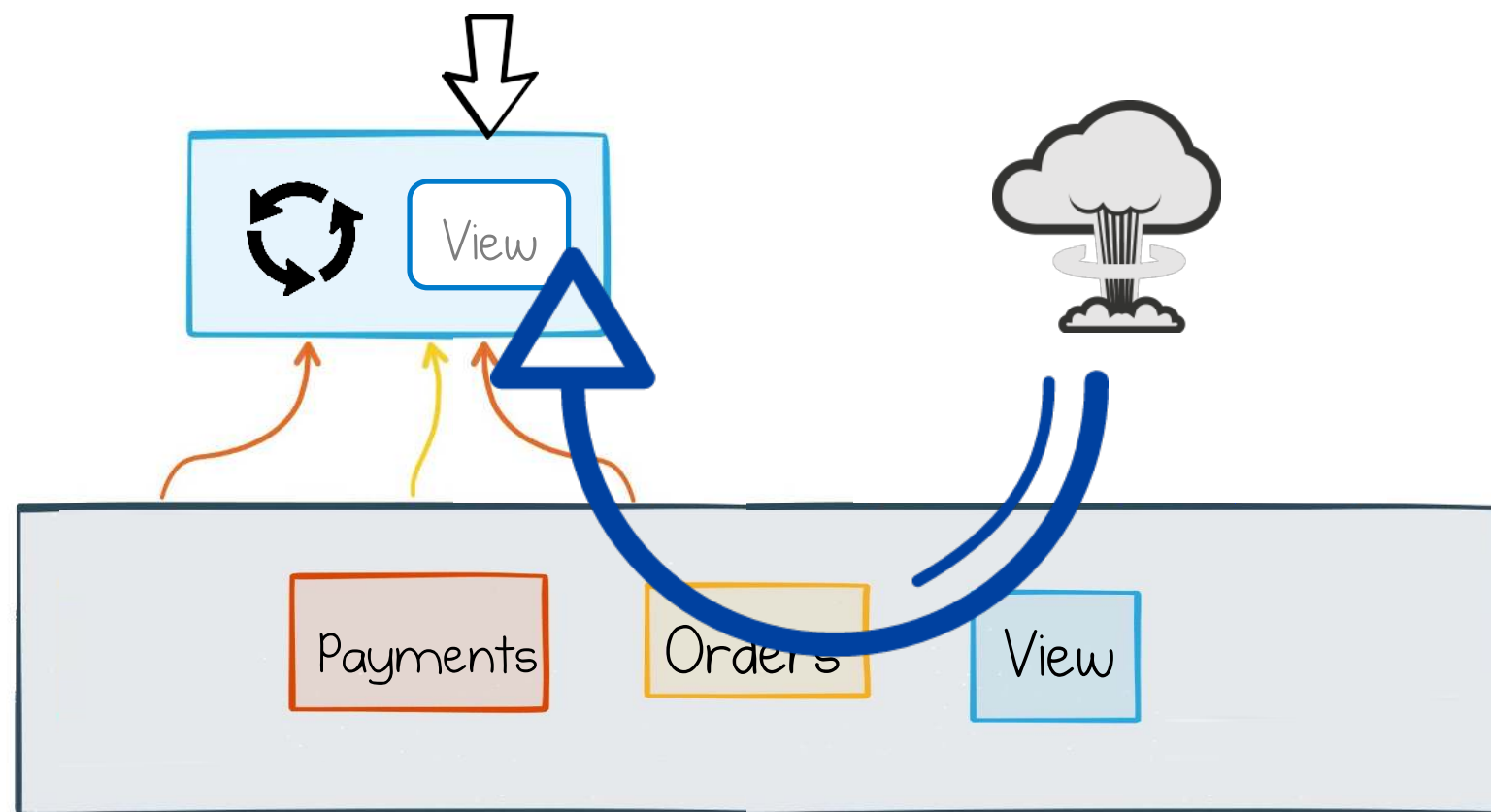
Clustering Reroutes Data to surviving node

Ownership of partitions is re-routed from dead node



But what about state?

"Cold" replica of state
takes over



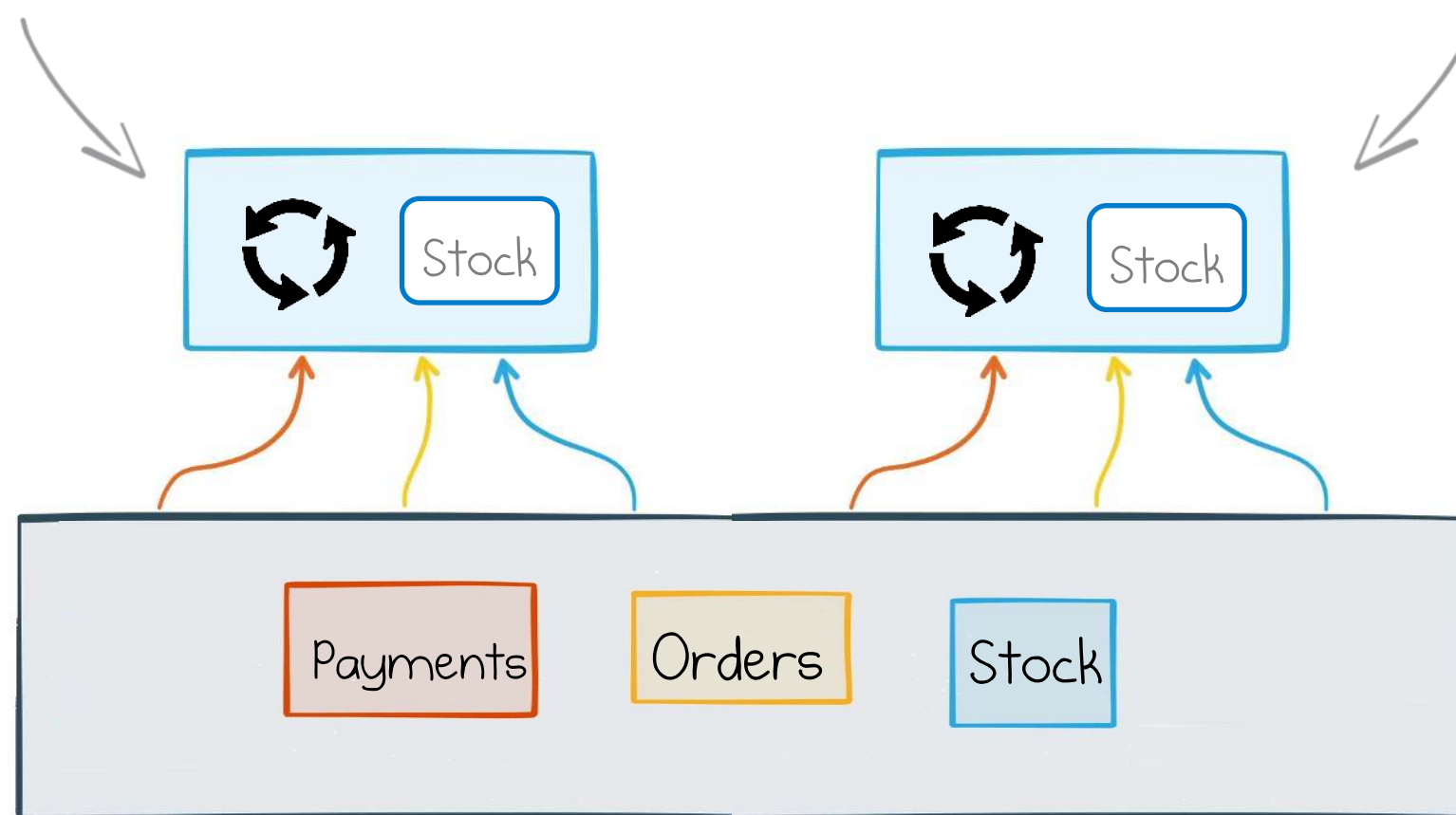
Processing & View

Storage Layer
(Kafka)

Primitives for sharding & replication

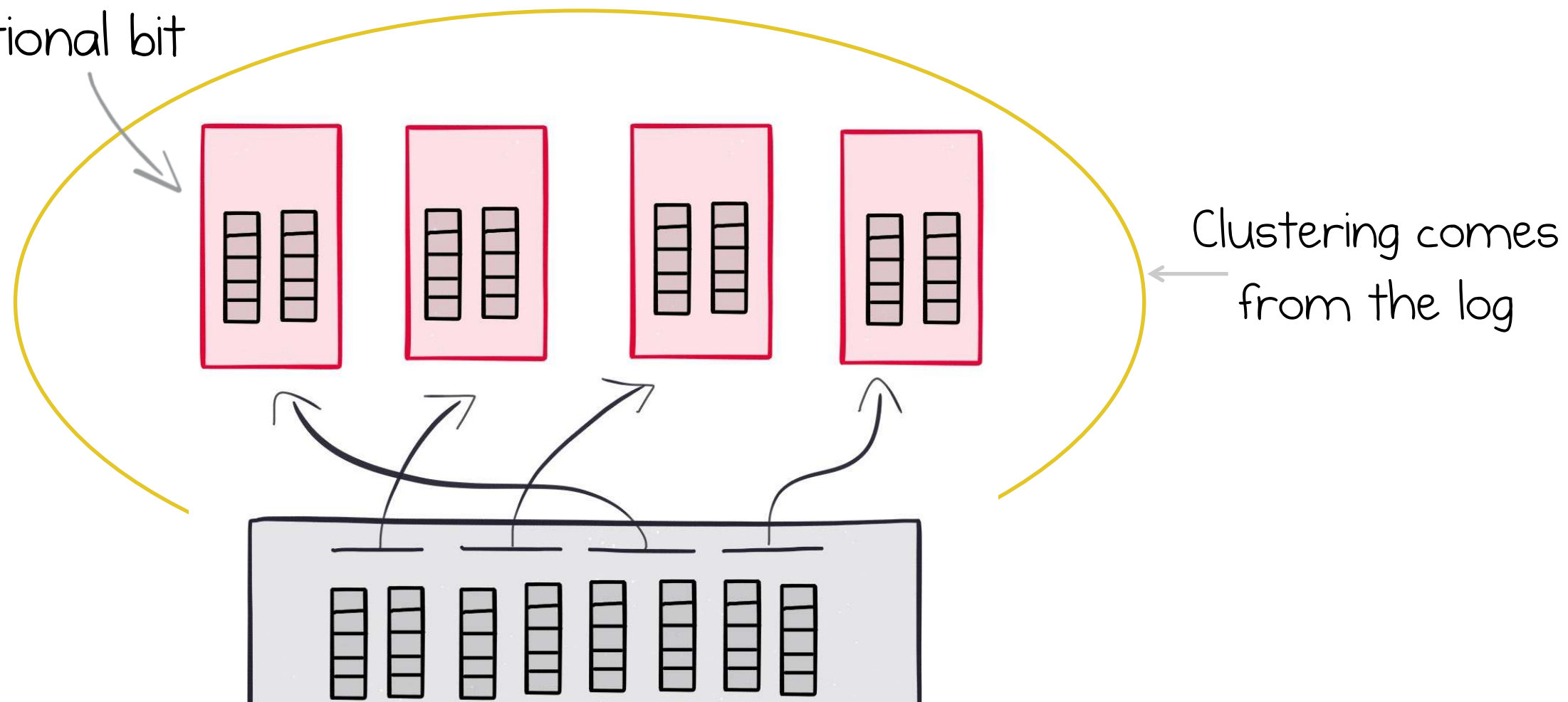
Sharding spread data over processors

Redundant copies are cached on other nodes

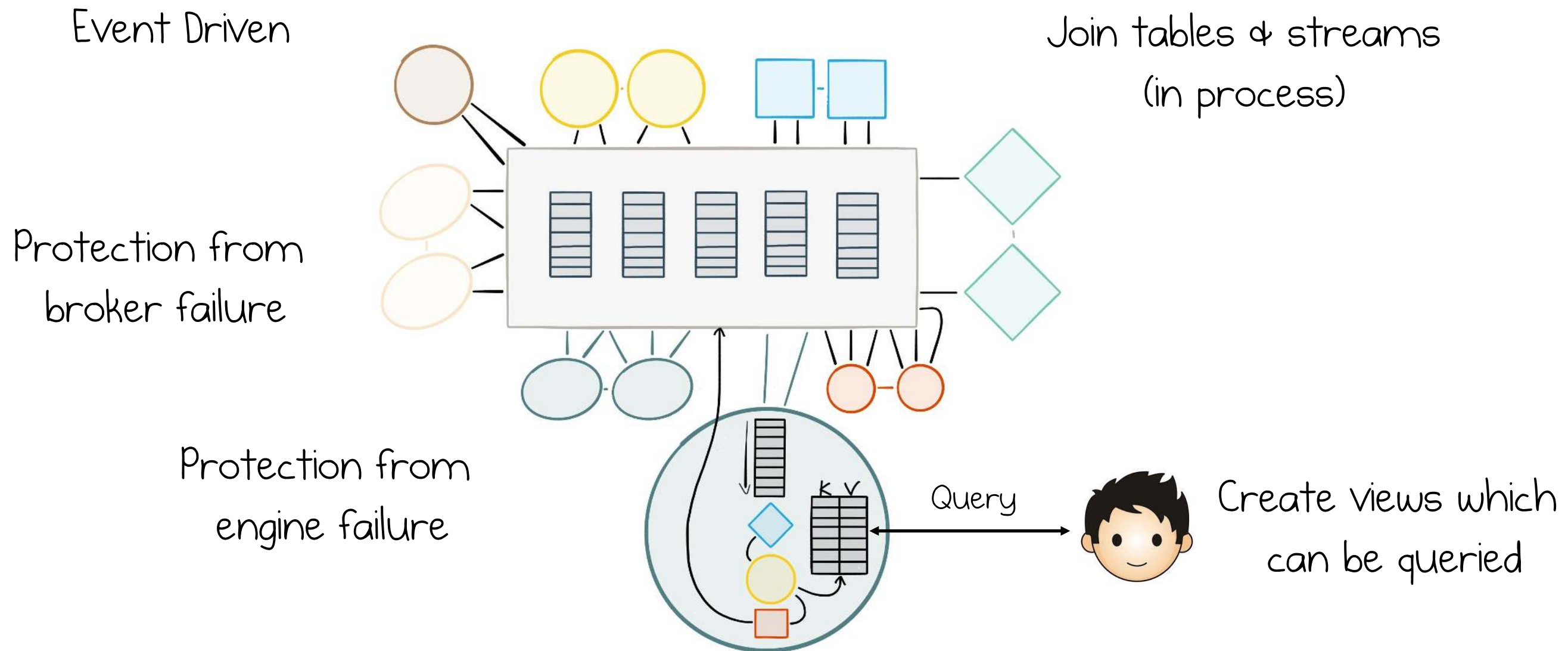


So processors inherit much from the log

You just write the functional bit

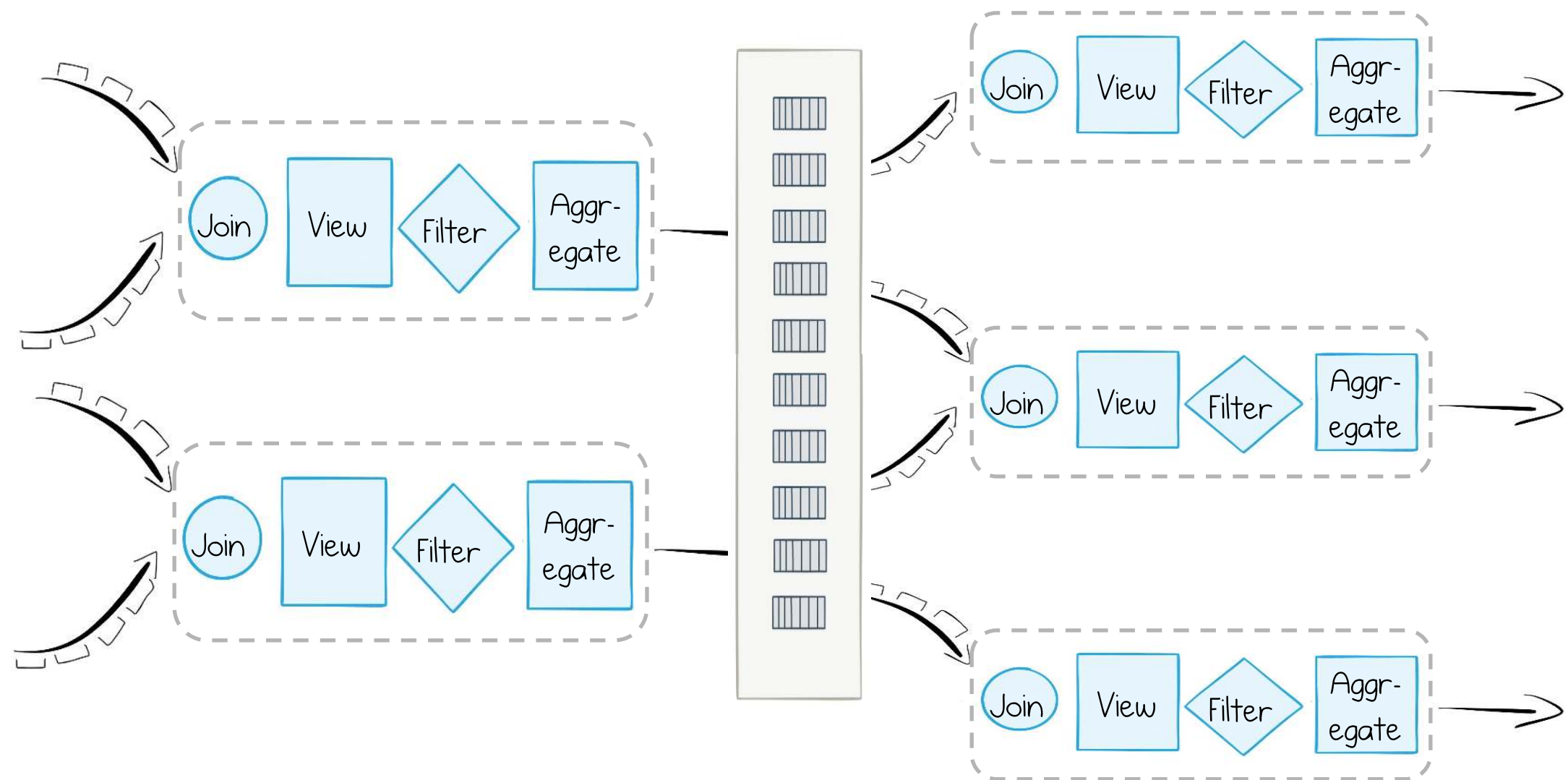


General framework for distributed, realtime data computation

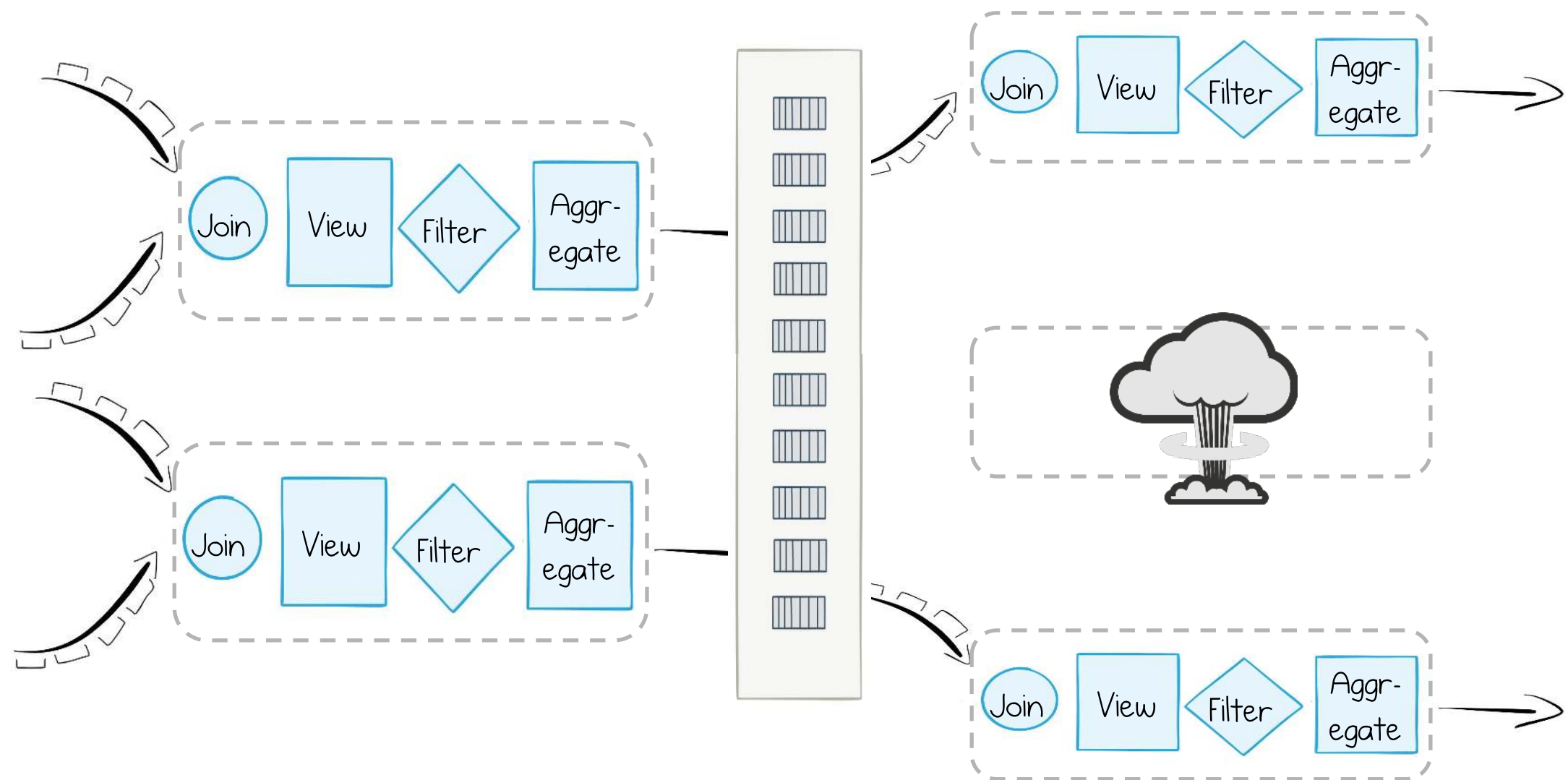


But stream
processing has a
problem

Correctness Guarantees in multi layer topologies

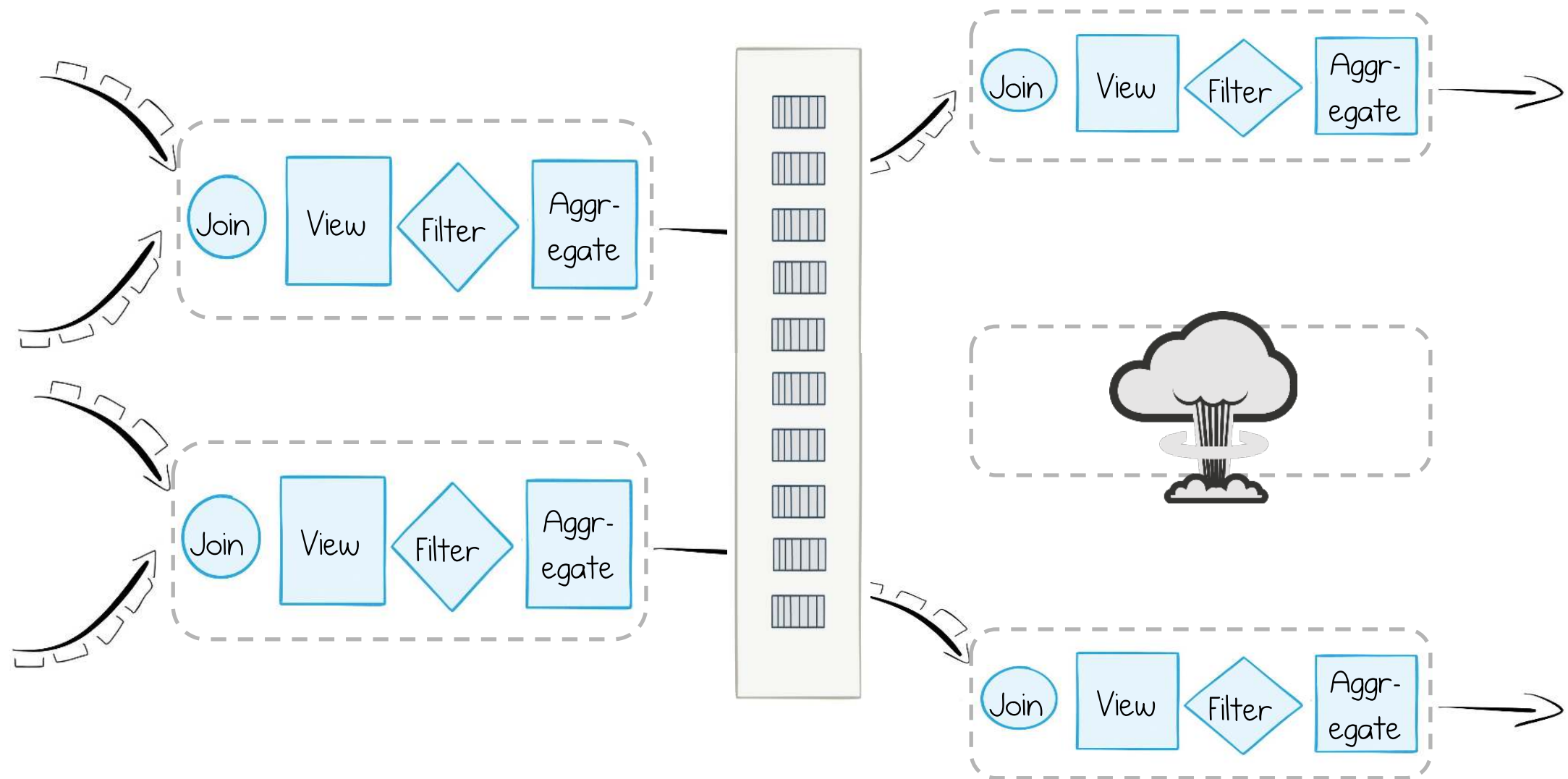


Data is rerouted, on failure, which can cause duplicates



Duplicates are a side effect of all at-least-once delivery mechanisms

Idempotence isn't enough

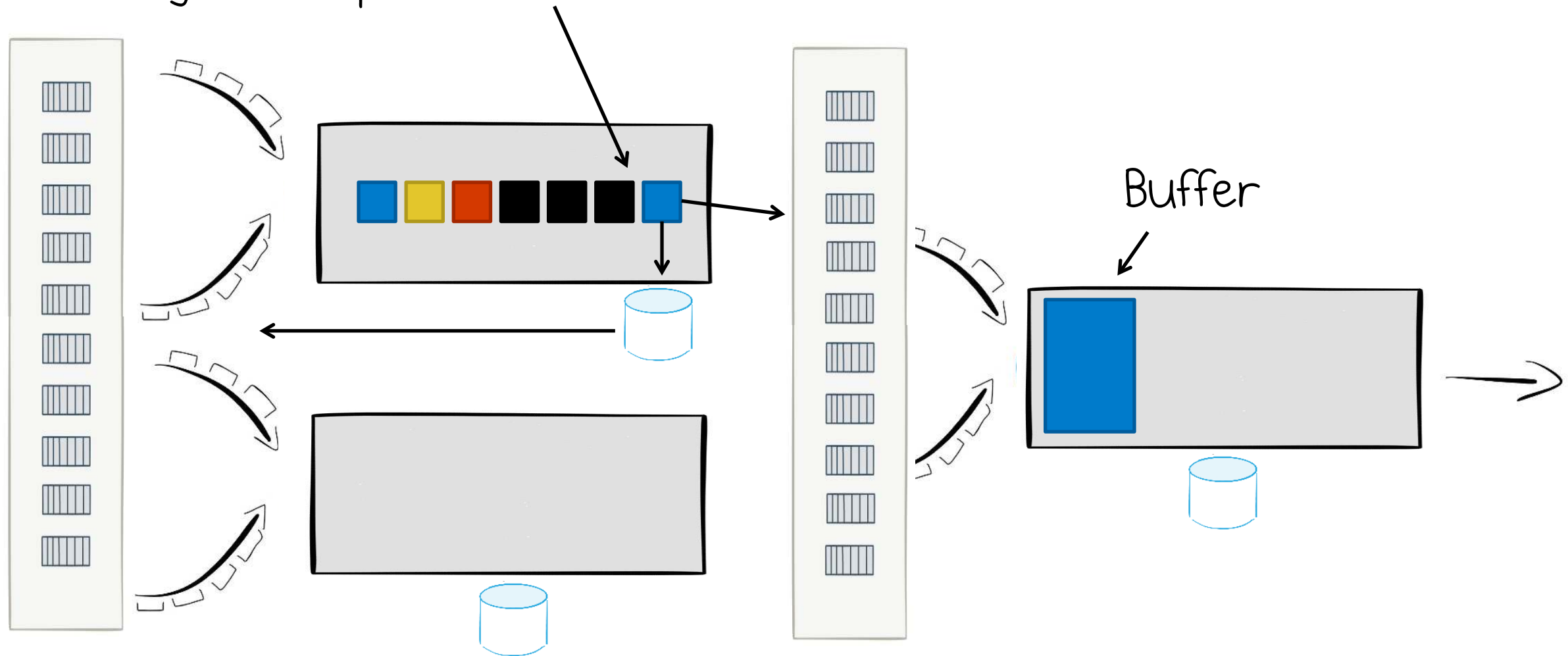


Distributed Snapshots^{*}

(transactions)

Transaction markers:

[Begin], [Prepare], [Commit], [Abort]



^{*}In development in Kafka

Chandy, Lamport - Distributed Snapshots: Determining Global States of Distributed Systems



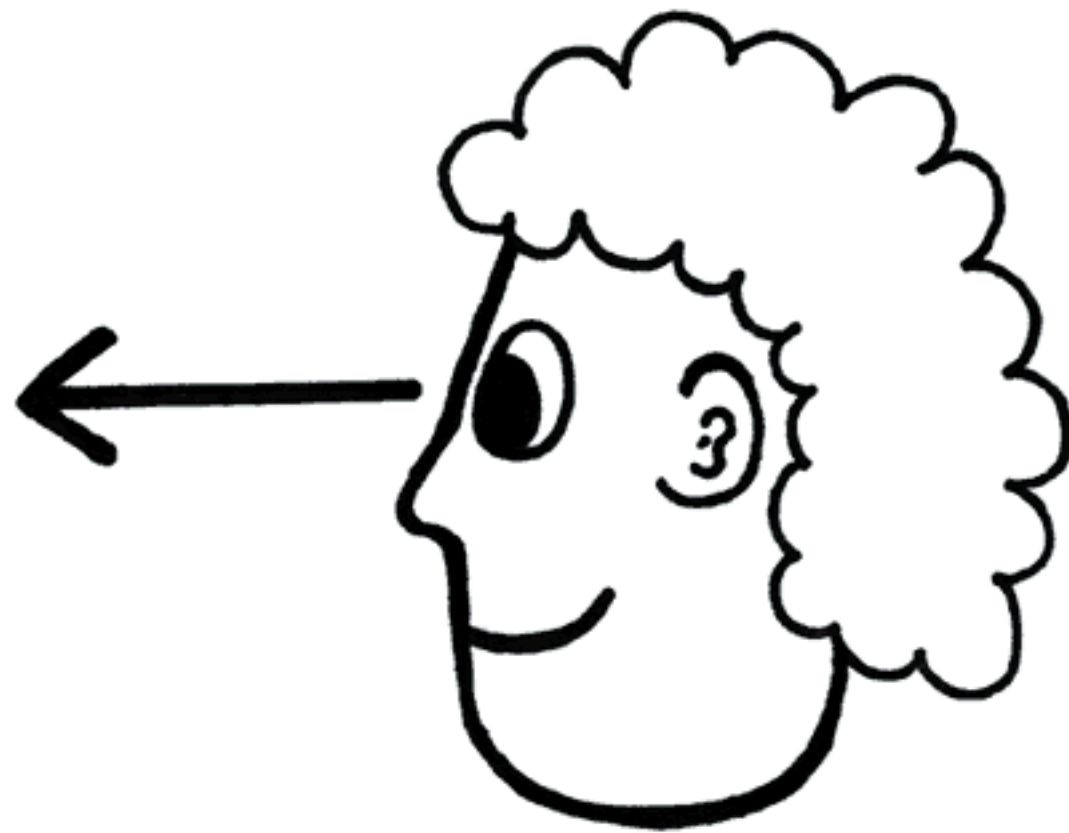
TAKE a
DEEP
Breath



So why use these
tools?

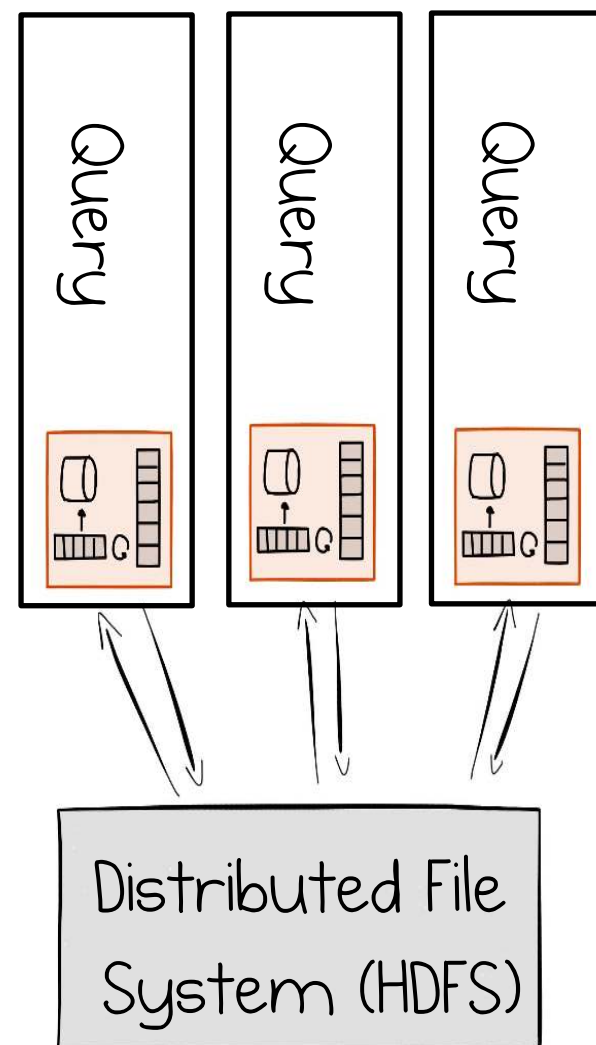
(1) Streaming is a
superset of batch

Databases look backwards

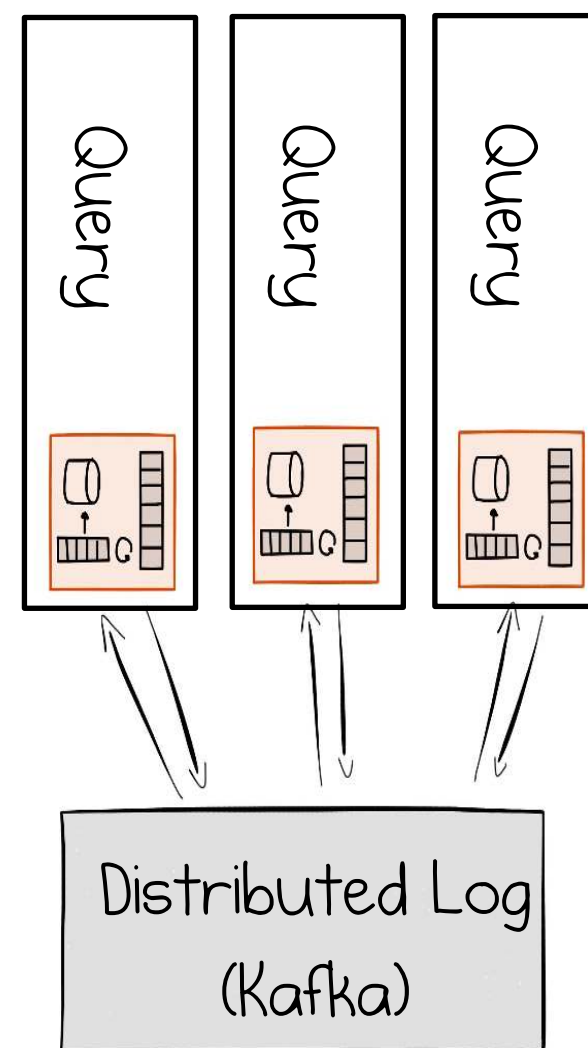


Batch == Streaming from offset 0

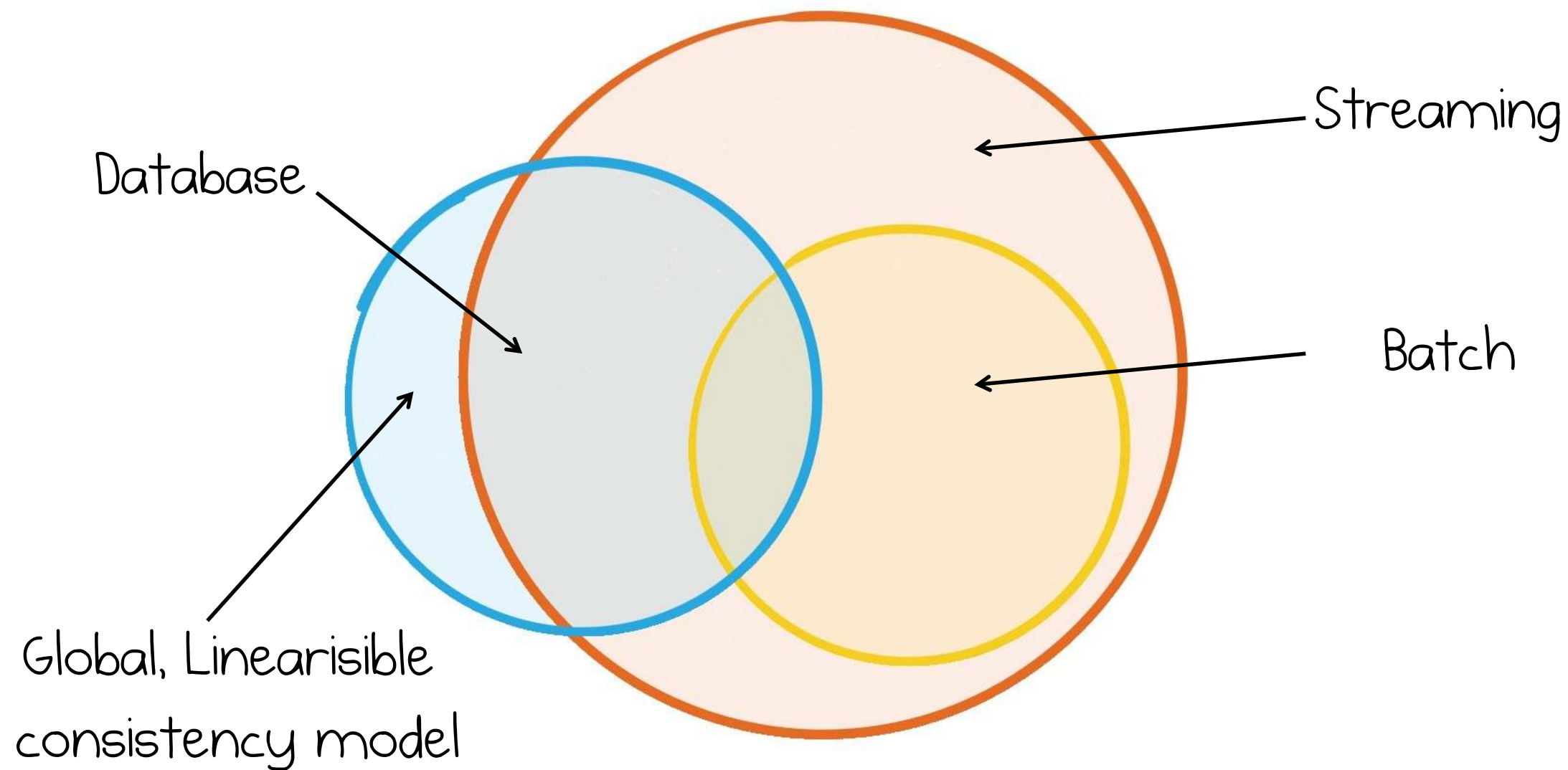
MPP Batch System



MPP Streaming System

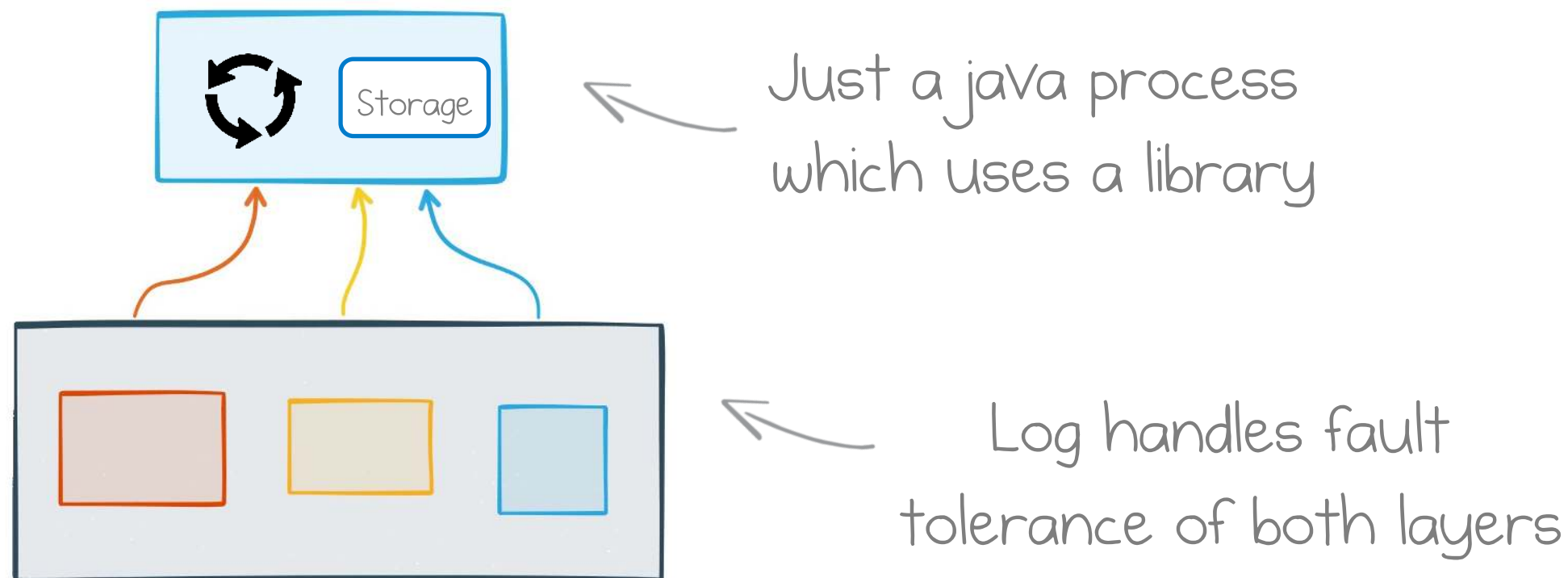


Streaming is the superset of batch

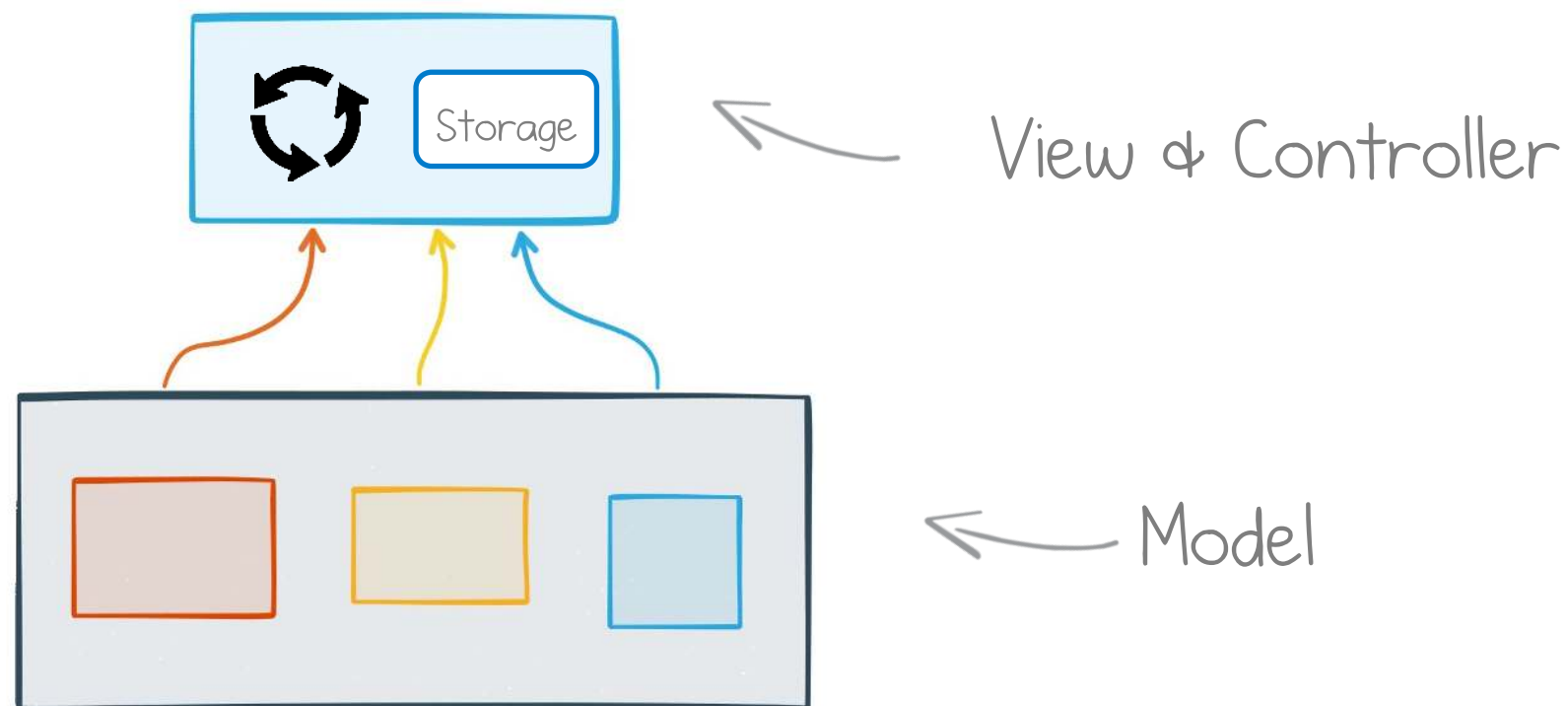


(2) Separates store & view

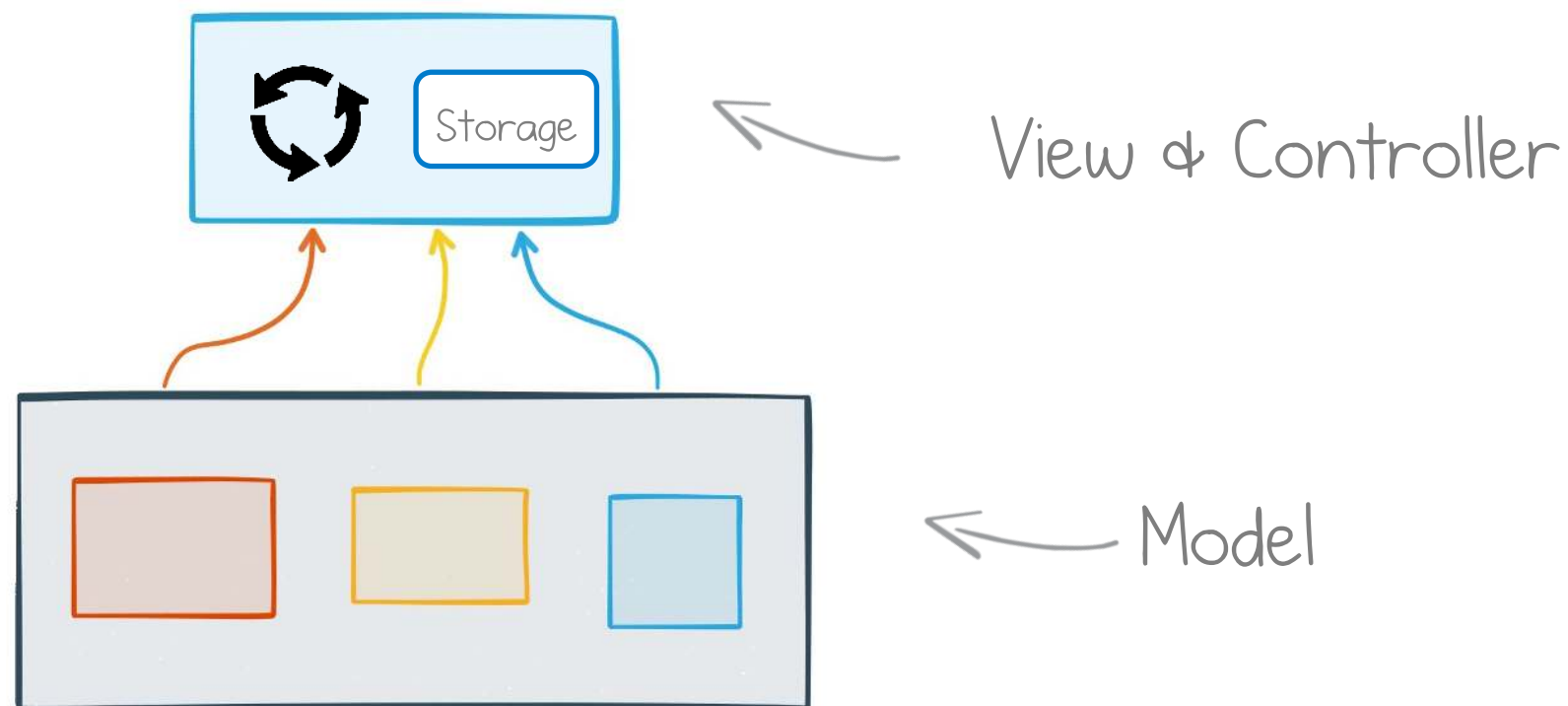
“Engine” part is lightweight but stateful



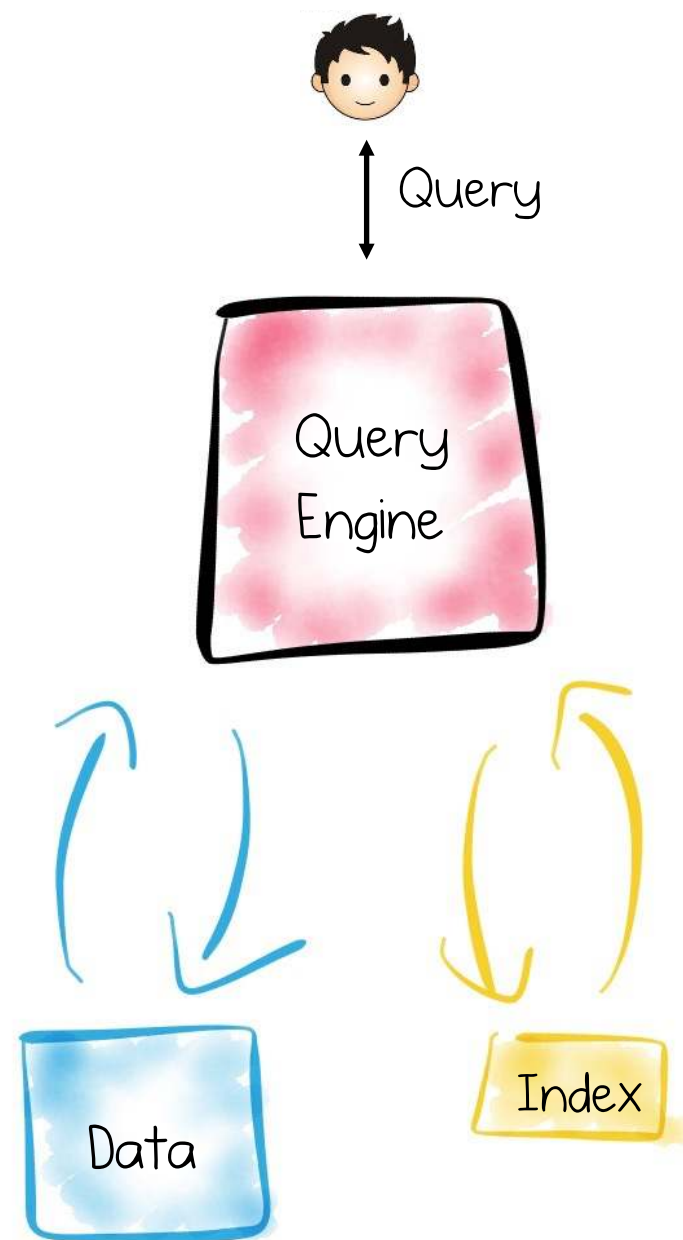
Separates Concerns of Model & View — Think MVC



Physically Separates Read & Write — Think CQRS

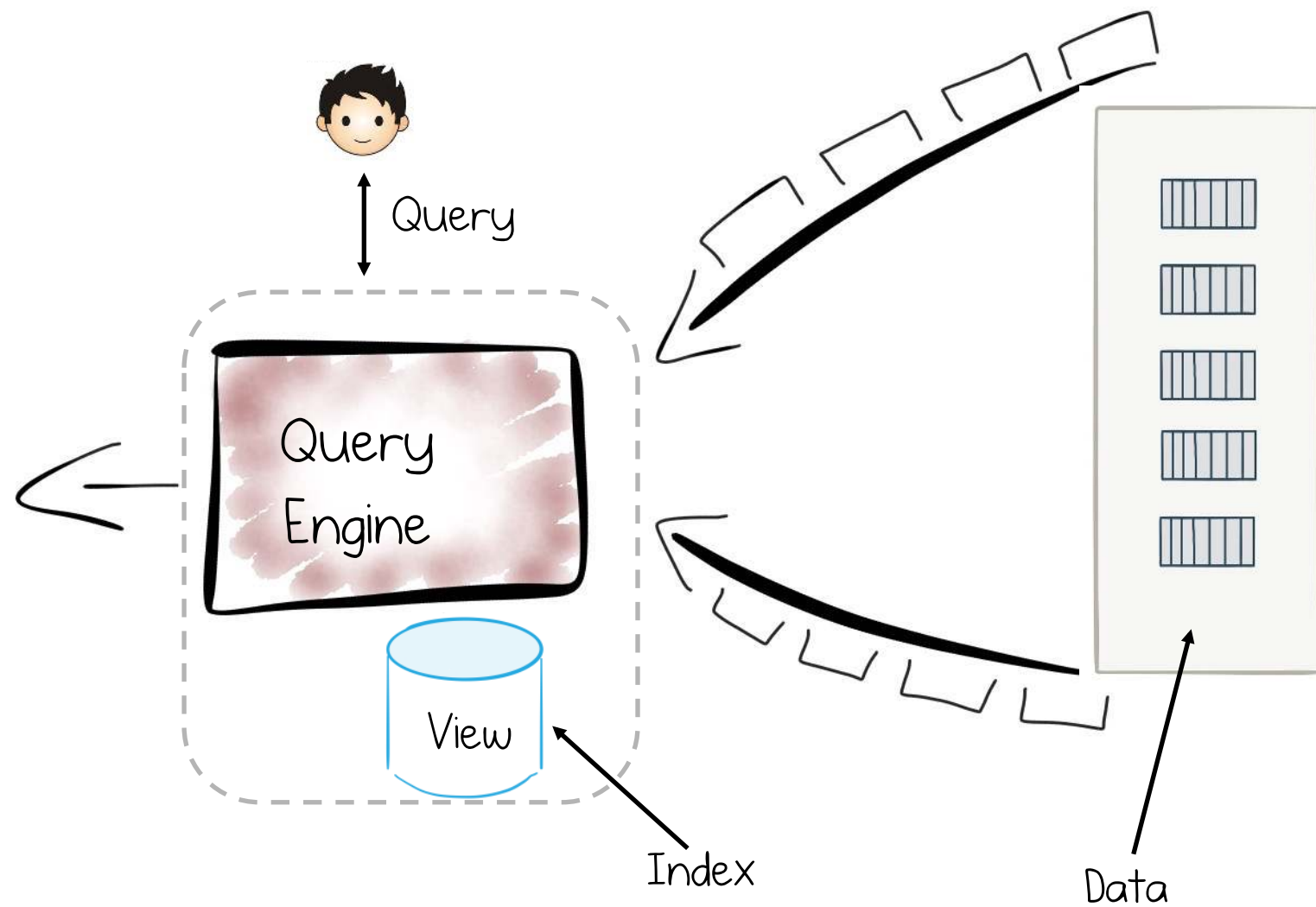


Database vs SSP



Database

vs

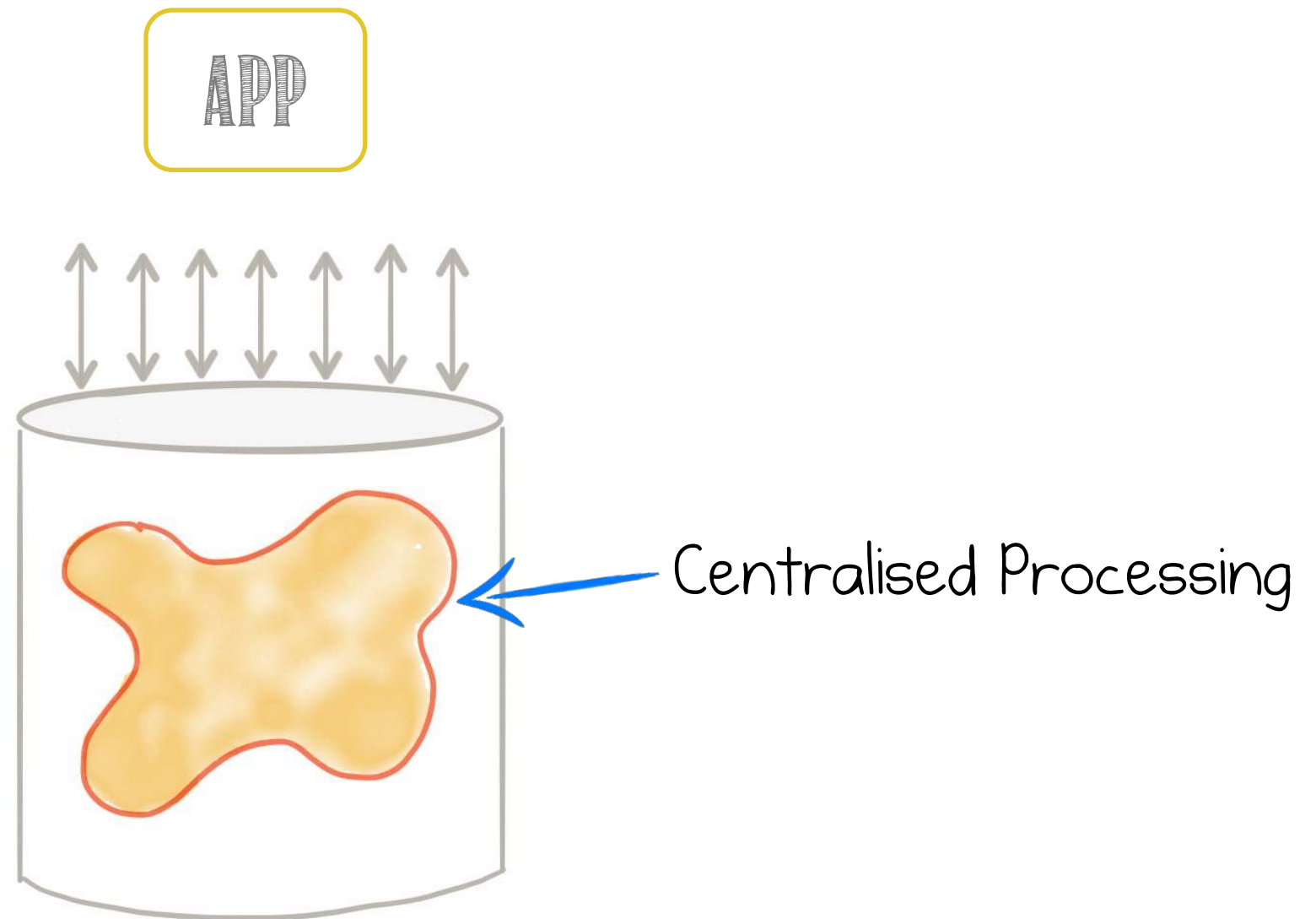


Stateful Stream Processor

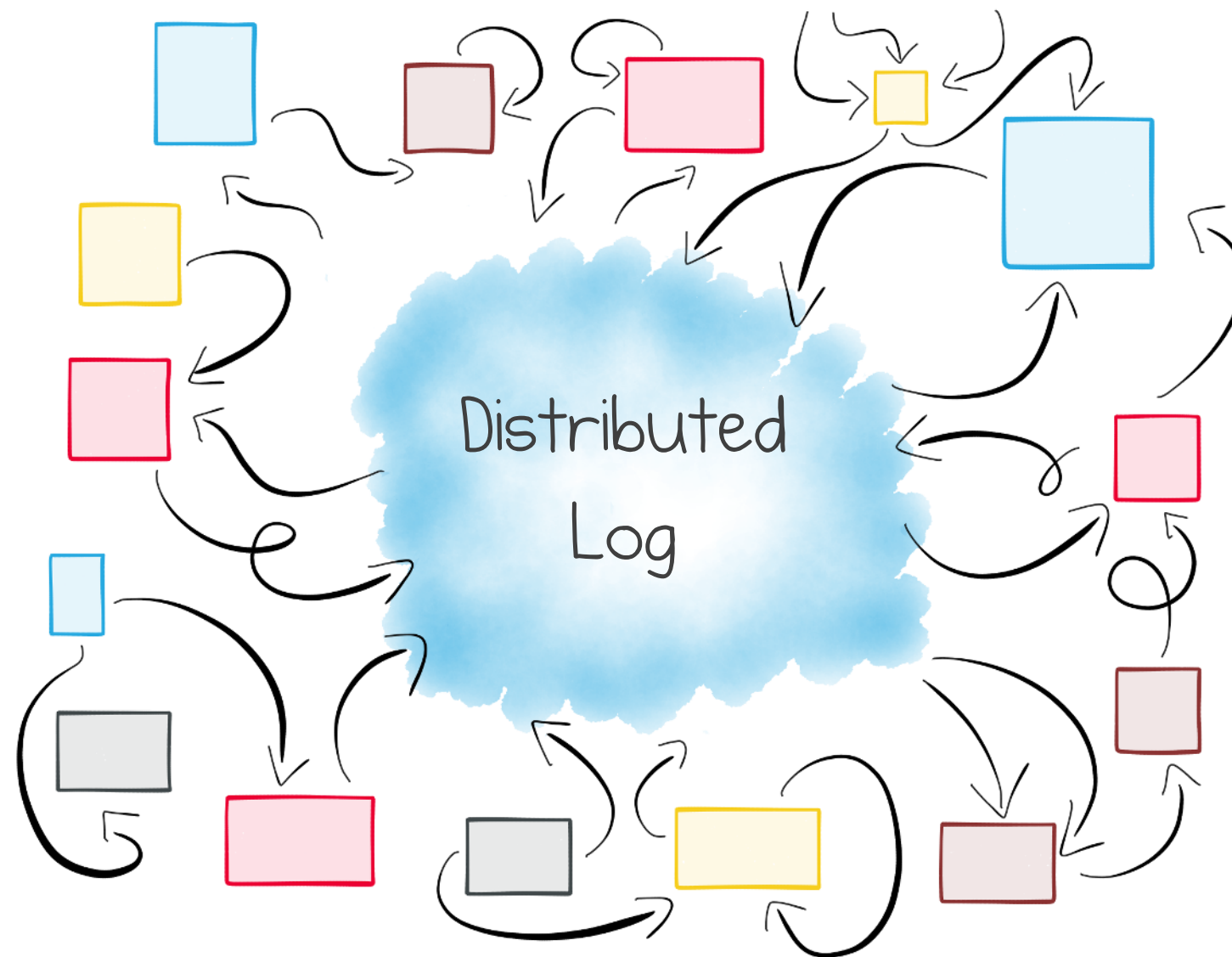
(3) Decentralised approaches
are more general

Rather than pushing processing into an “appliance”

(code -> data)



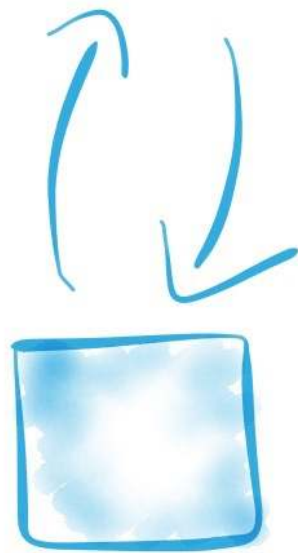
Decentralised Processing over many user-specific views



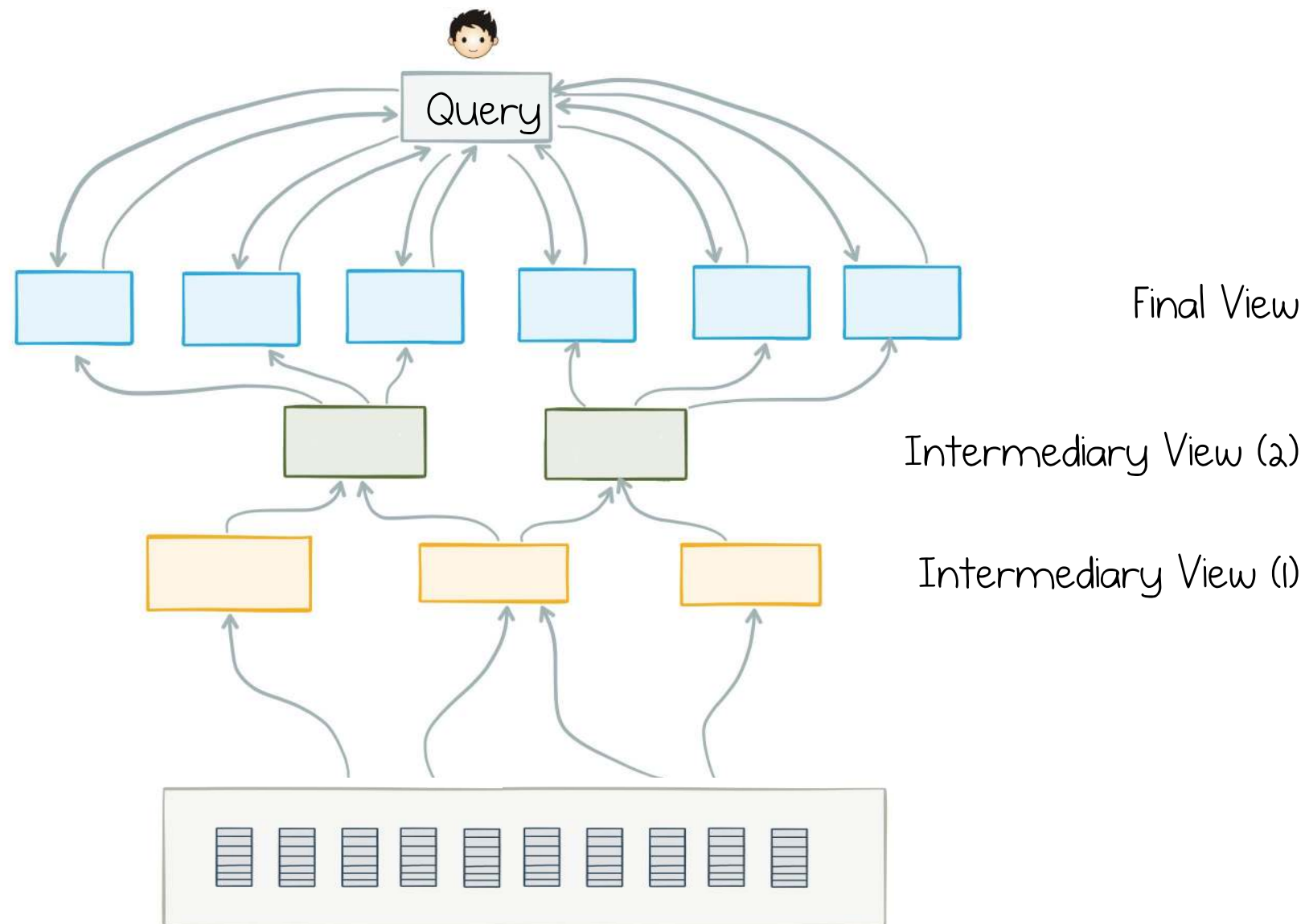
Data Decentric Architecture

This more general
than than just
analytics use cases

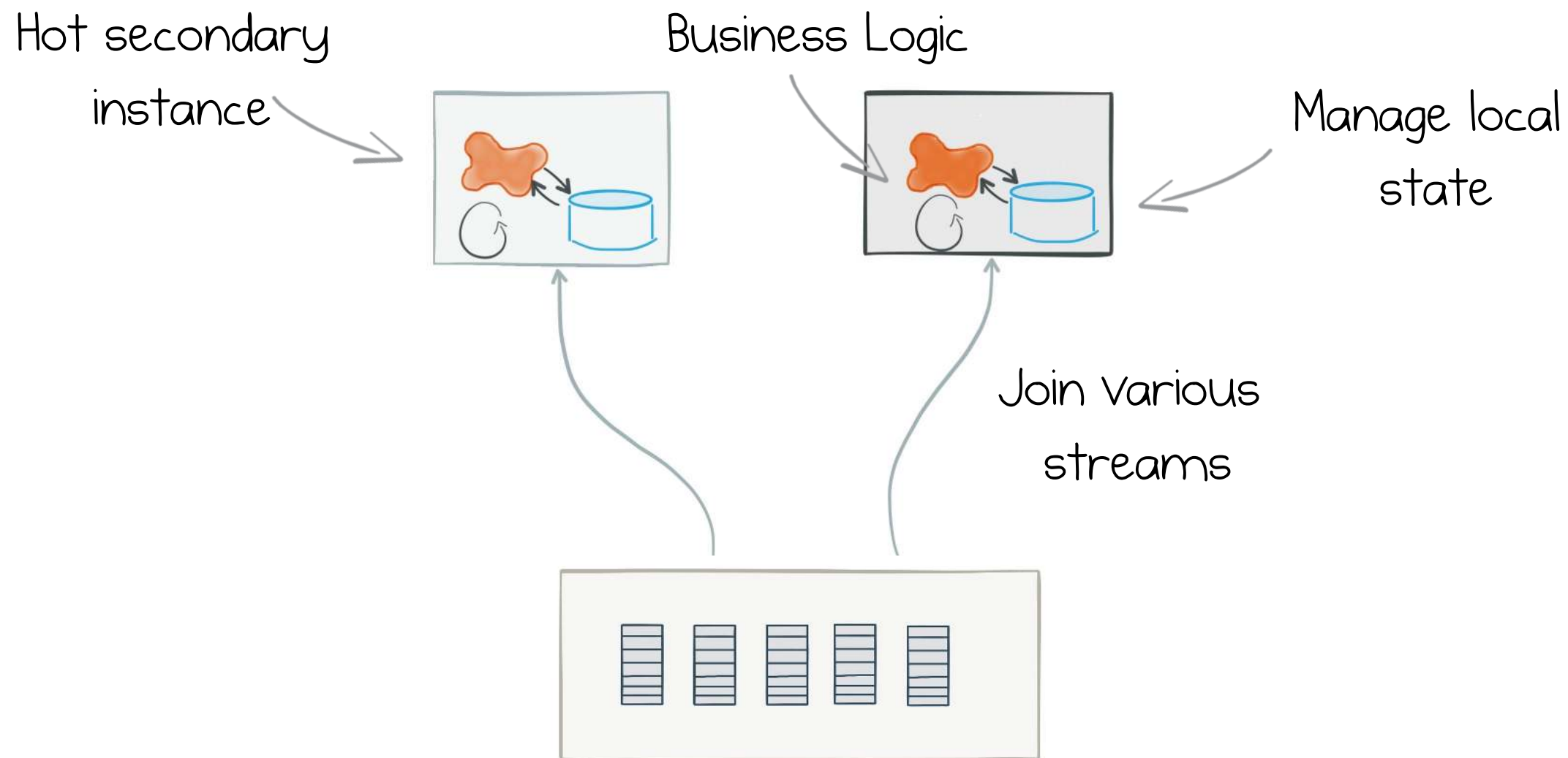
It's more than taking a
database and adding push
notifications



Whether you're building a hulking, multistage, analytic platform



Or a simple microservice that needs to run hot-hot & scale



Composable Primitives

Traditional DB

Query
Engine

Distributed DB

Replication

Distributed Systems

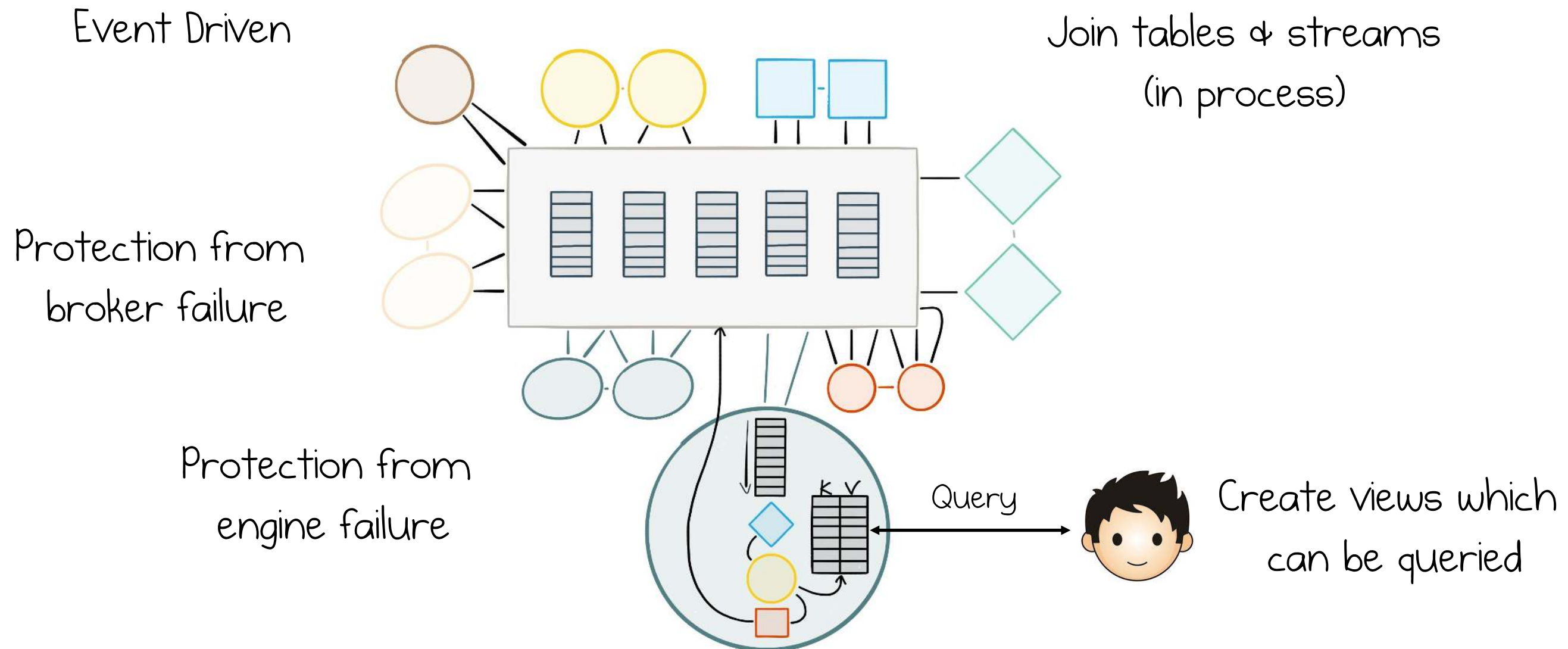
Membership

Global
Consistency

Sharding

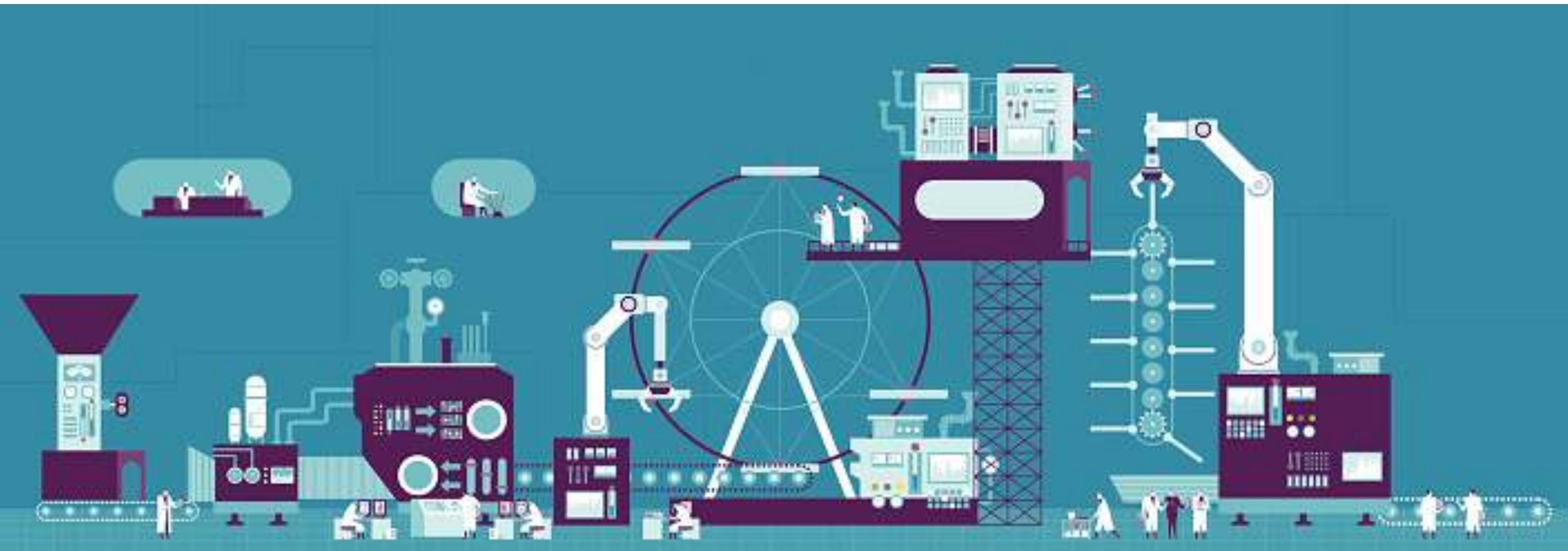
Work
Distribution

General framework for distributed, event-driven data computation



Stateful Stream Processing

Framework for building a streaming data systems, just for you “~)



Find out more:

- <http://www.confluent.io/blog/introducing-kafka-streams-stream-processing-made-simple/>
- <https://martin.kleppmann.com/2015/02/11/database-inside-out-at-salesforce.html>
- http://cidrdb.org/cidr2015/Papers/CIDR15_Paper16.pdf
- <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/cidr07p42.pdf>
- <http://highscalability.com/blog/2015/5/4/elements-of-scale-composing-and-scaling-data-platforms.html>
- <https://speakerdeck.com/bobbycalderwood/commander-decoupled-immutable-rest-apis-with-kafka-streams>
- <https://timothyrenner.github.io/engineering/2016/08/11/kafka-streams-not-looking-at-facebook.html>
- <https://www.madewithtea.com/processing-tweets-with-kafka-streams.html>
- <http://www.infolace.com/blog/2016/07/14/simple-spatial-windowing-with-kafka-streams/>
- <http://www.slideshare.net/Zacharycox/updating-materialized-views-and-caches-using-kafka>

The end

@benstopford

<http://benstopford.com>